

SoC 를 위한 *Peripheral* 설계

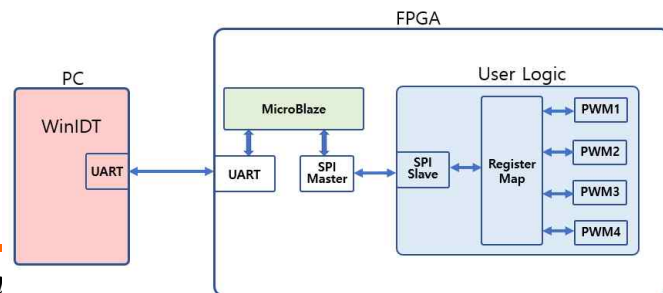
Reference : MicroBlaze.v15 [IHIL]

2025-06-24

Table of Contents

➤ SoC를 위한 Peripheral 설계

1. Xilinx IP
2. Create and Package New IP
3. SPI
 - 1) SPI Master
 - 2) SPI Slave
 - 3) SPI Controller
4. UART
5. AMBA
6. **MicroBlaze_Hello World**
7. MicroBlaze_LED_Counter
8. MicroBlaze_Peripheral Implementation
9. MicroBlaze_User Logic Interface

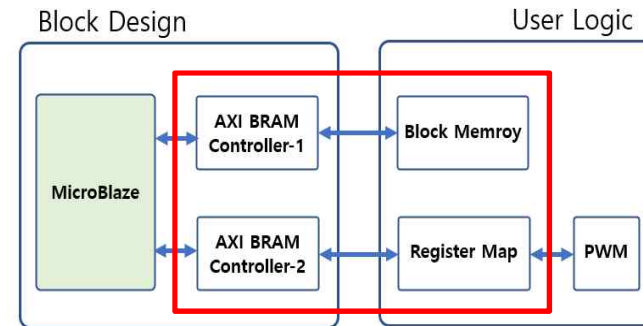


10. SPI_Master_IP(MicroBlaze_User Logic Interface)

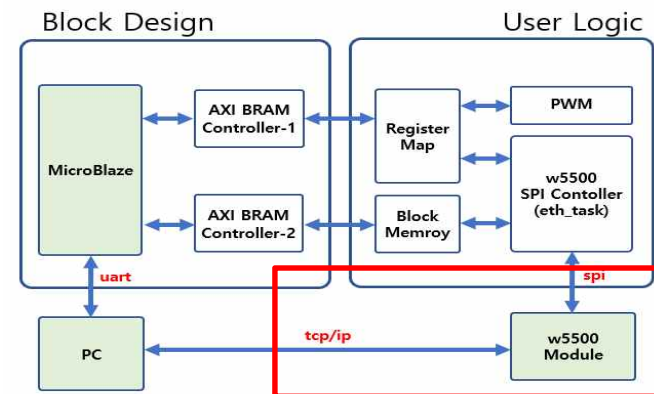
11. TCP_IP Implementation Using W5500

12. MicroBlaze_Block Memory Interface-1

13. MicroBlaze_Block Memory Interface-2



14. w5500 Interface Implementation



➤ SoC Peripheral RTC Design Project

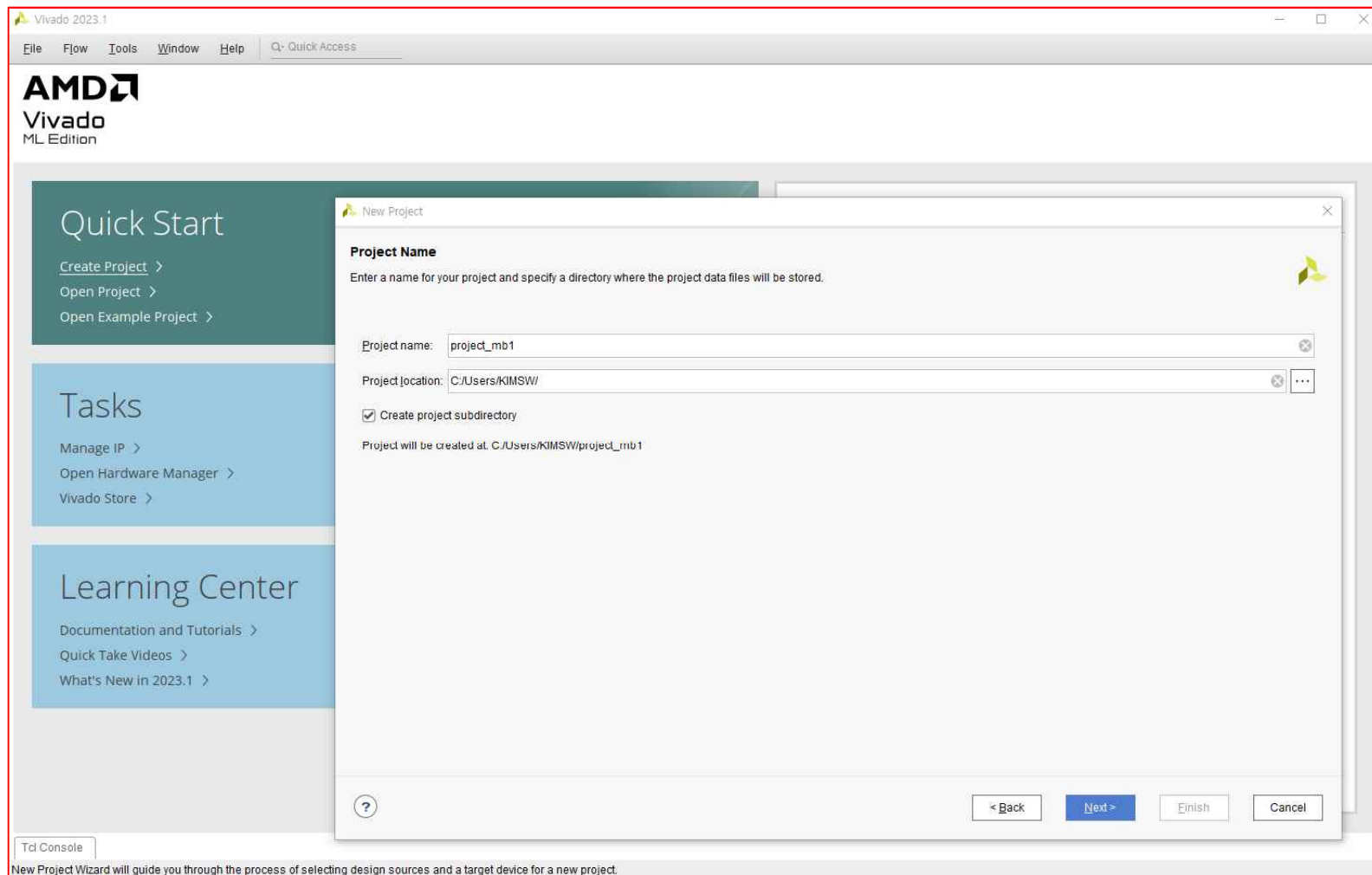
- *MicroBlaze*를 이용하여 *Hello World* 출력
 - *MicroBlaze*는 *Processor Core*와 *Peripheral* 이 분리
 - 원하는 *Peripheral* 추가 및 개수도 필요한 만큼 추가하여 사용가능
 - *MicroBlaze Processor Core*와 *Peripheral* 중에 *UART* 를 추가하여 *UART* 포트를 통하여 *Hello World* 를 출력 기능 구현
- *FPGA*에서 *MicroBlaze*를 사용하는 목적
 - 사용자가 만든 *User Logic*을 제어하고, *Logic* 으로 구현할 수 없는 부분을 *MicroBlaze* 로 구현
 - *FPGA*에서 *IP*와 *User Logic* 을 생성해서 *MicroBlaze*사용하는 궁극적인 목적은 *HW*로 구현하는 부분과 *SW*로 구현하는 부분을 나누어서 구현하고 서로 간에 인터페이스를 구현하여 외부에 추가적인 부품을 사용하지 않는 *SoC (System on Chip)*를 구현

Hello World Implementation

Microblaze
mb2.xpr//microblaze_0.xpr

➤ Hello World Implementation

- Vivado 2023.1 실행 → [Create Project] 클릭 → [Next]
- Project name 및 위치 설정 → Next

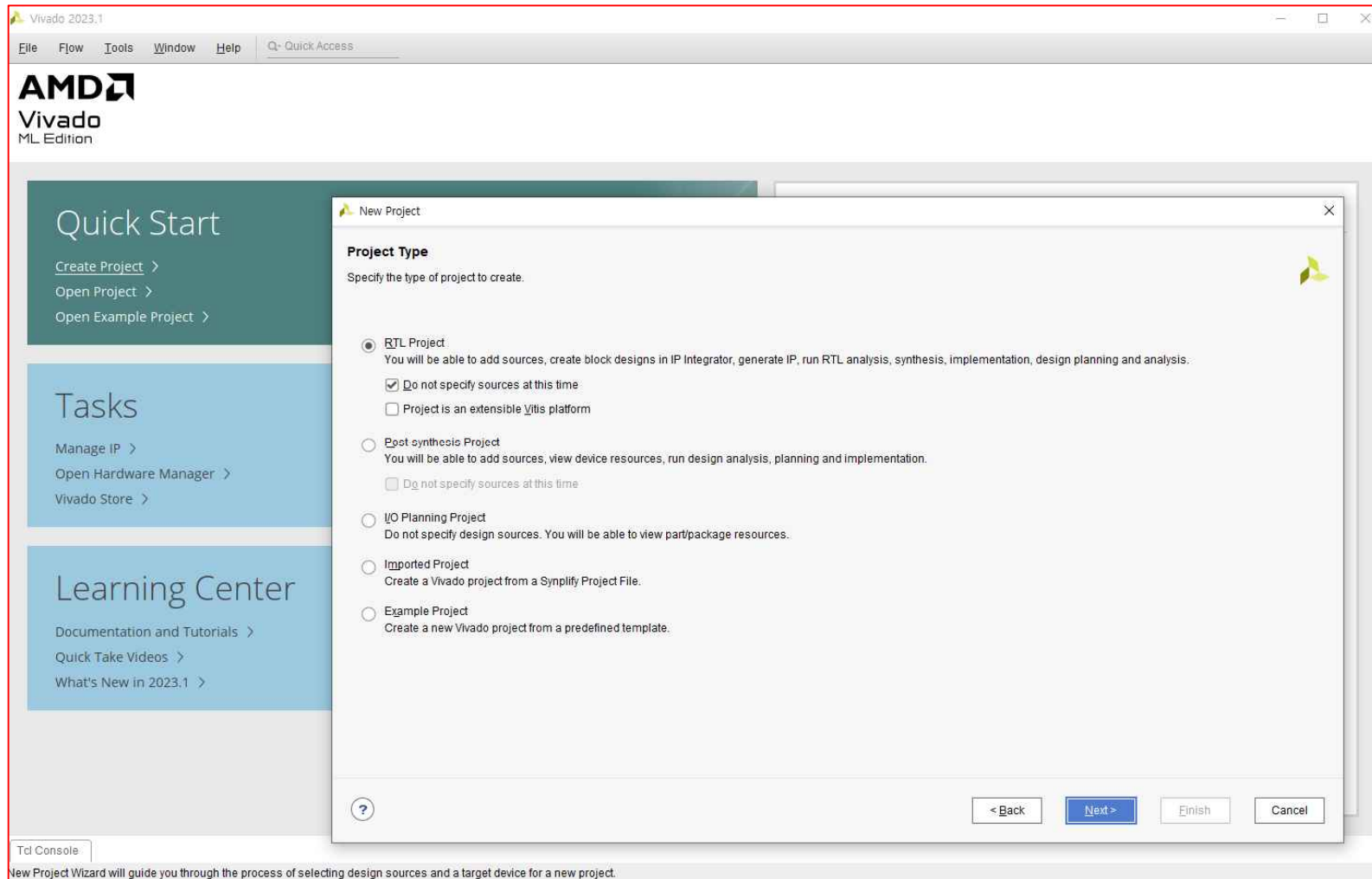


Hello World Implementation

Microblaze
mb2.xpr//microblaze_0.xpr

➤ Hello World Implementation

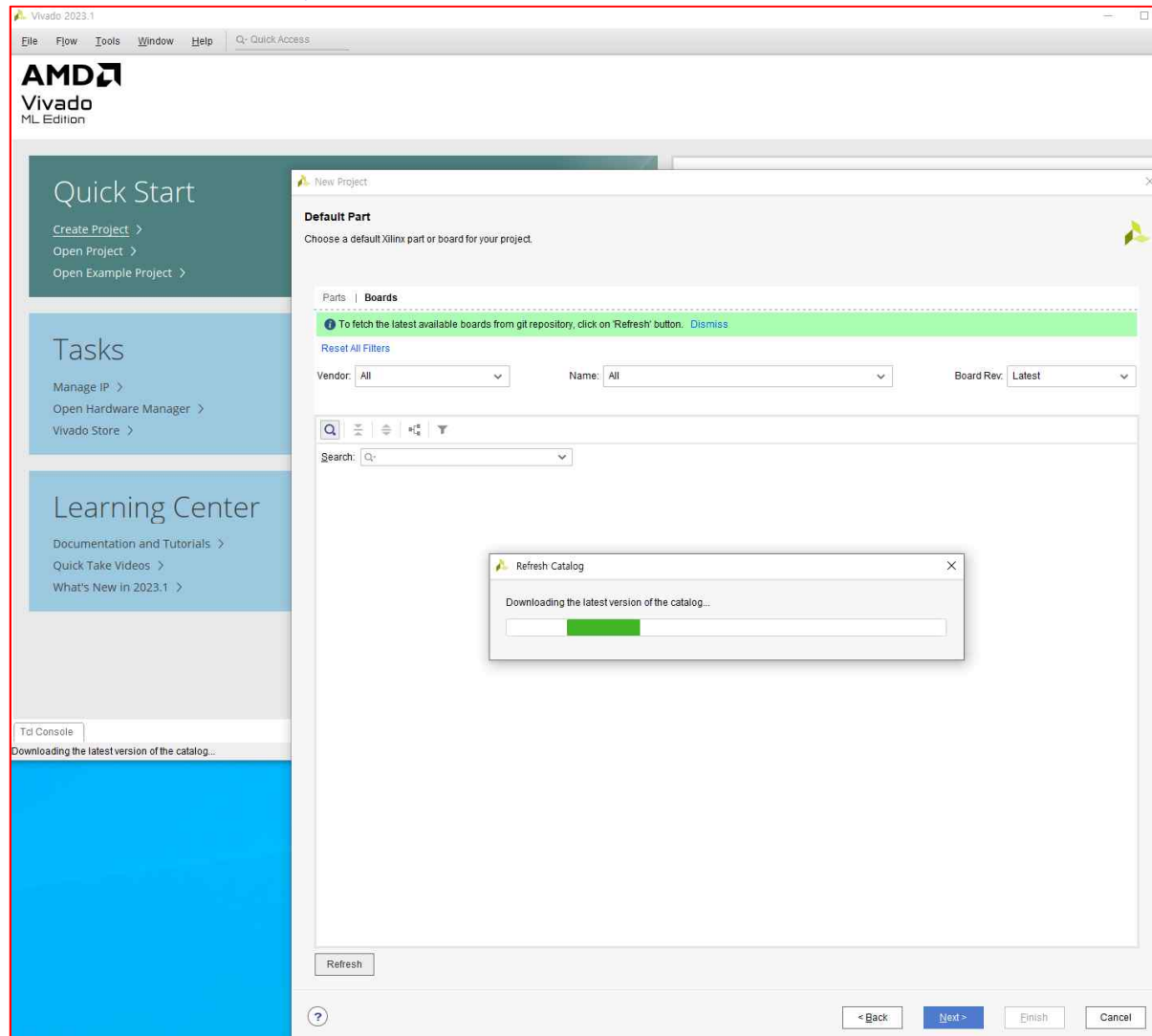
- RTL Project → Next



Hello World Implementation

Microblaze

- **Hello World Implementation**
 - **Default Part ➔ Boards ➔ Basys3 ➔ Next**

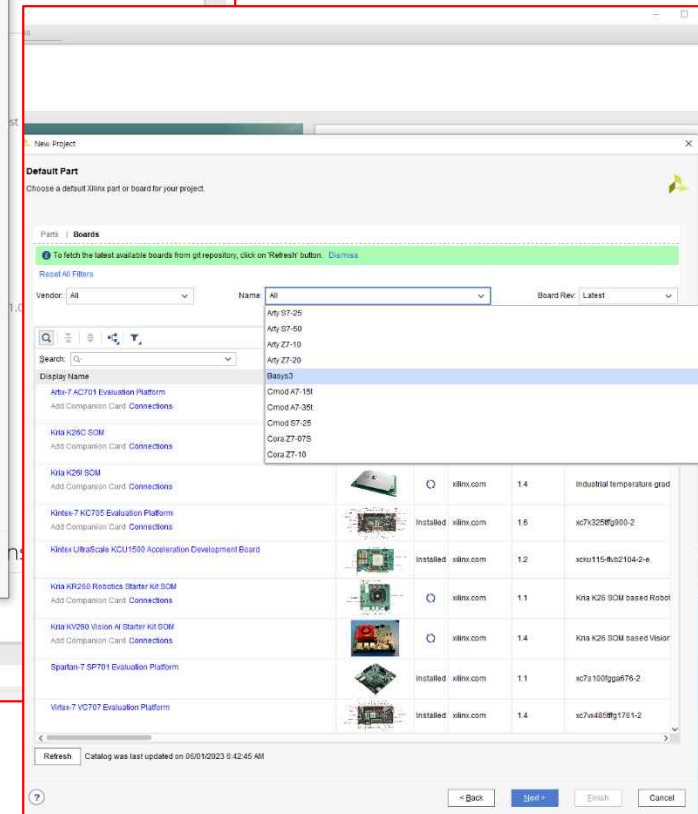
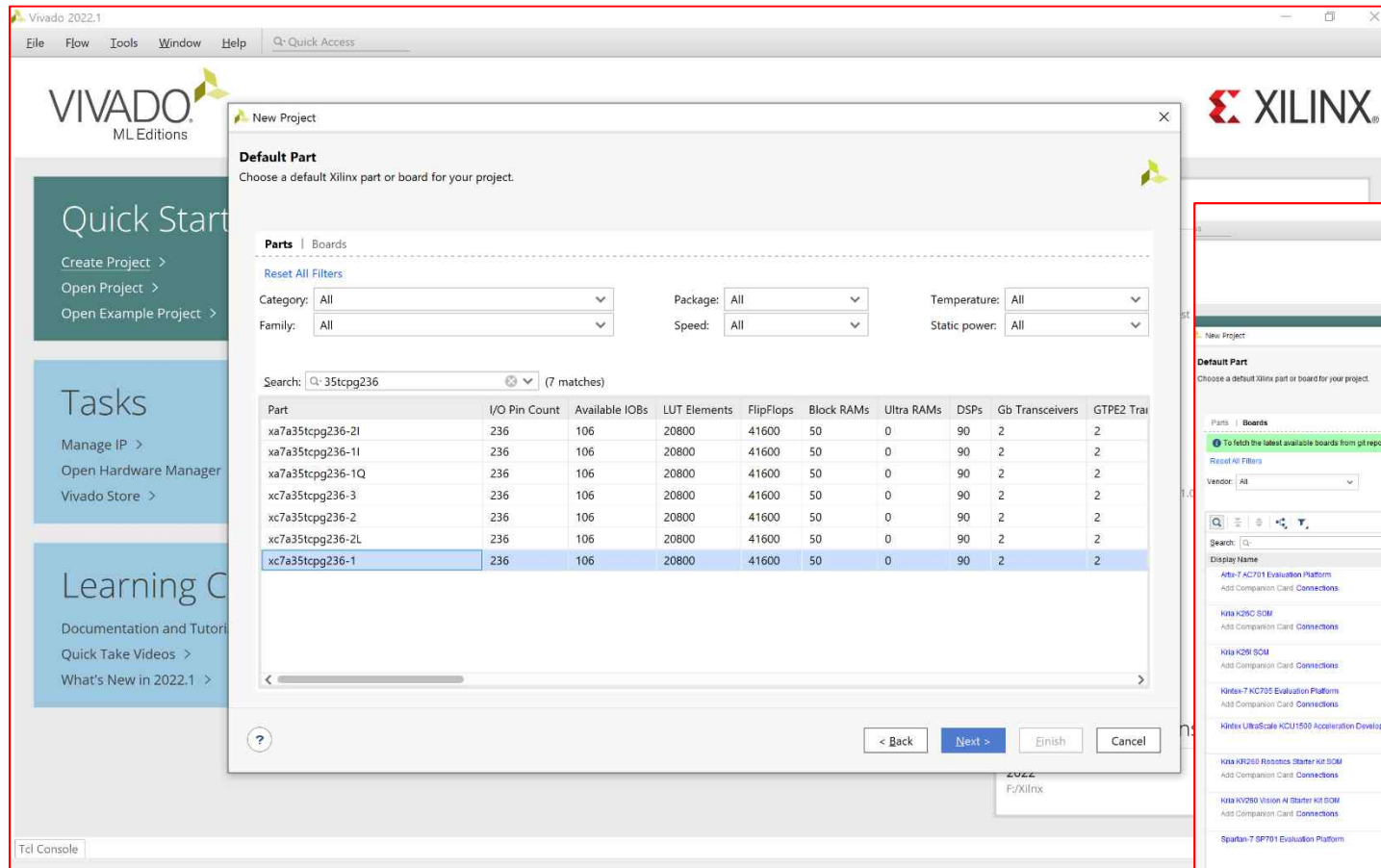


LED_Counter Implementation

➤ LED_Counter Implementation ➔ Basic Template Implementation

▪ Default Part ➔ Parts ➔ xc7a35tcpg236-1 ➔ Next

✓ [or] Default Part ➔ Boards ➔ Basys3 ➔ Next

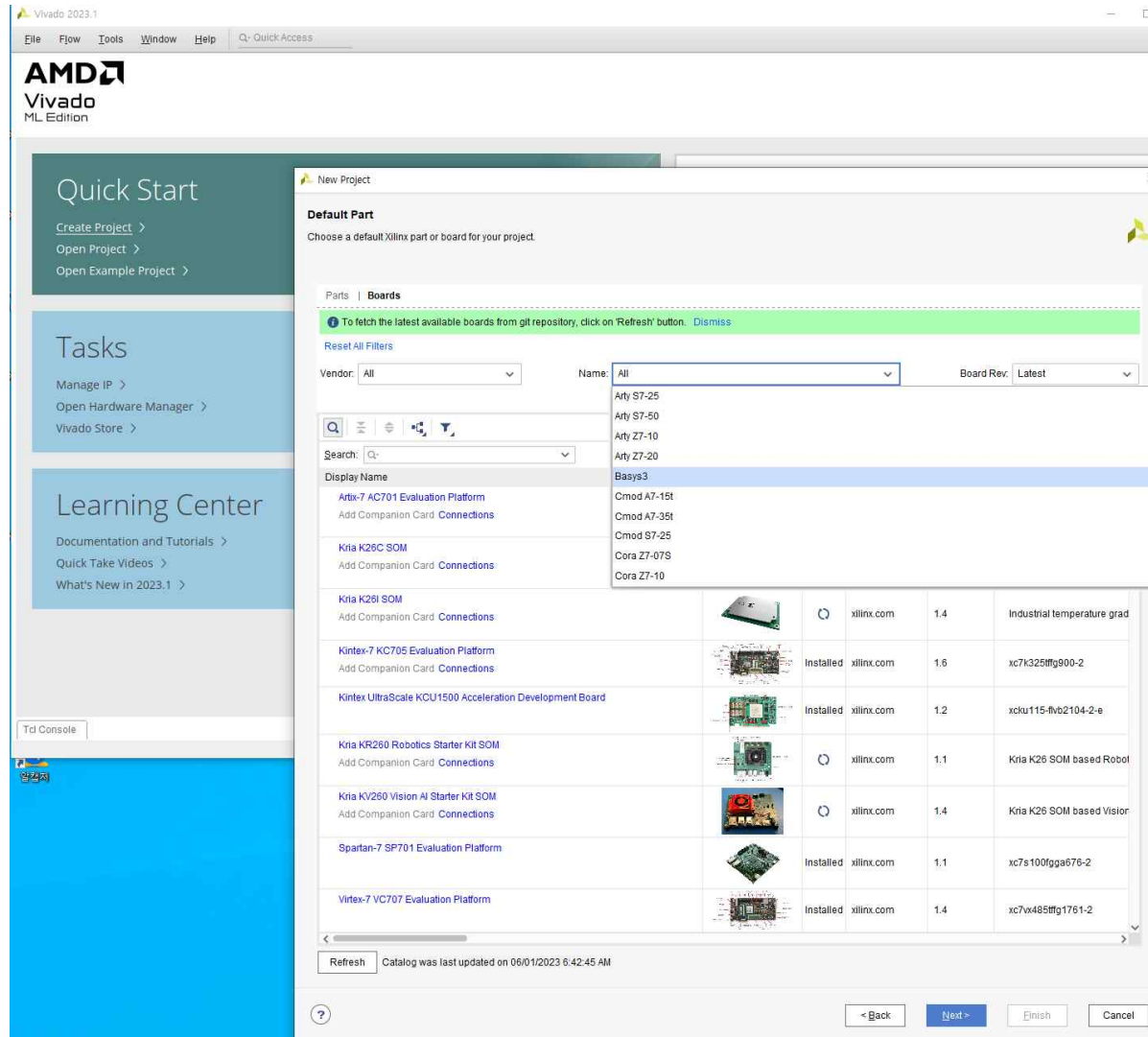


Hello World Implementation

Microblaze

➤ Hello World Implementation

- Default Part ➔ Boards ➔ Basys3 ➔ Next



Hello World Implementation

Microblaze

➤ Hello World Implementation

- Default Part ➔ Boards ➔ Basys3 ➔ Next

New Project

Default Part
Choose a default Xilinx part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

Vendor: All | Name: Basys3 | Board Rev: Latest

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count	Board Rev
Basys3			digilentinc.com	1.2			

[Install](#)

[Refresh](#) Catalog was last updated on 06/01/2023 6:42:45 AM

[? < Back](#) [Next >](#) [Enish](#) [Cancel](#)

New Project

Default Part
Choose a default Xilinx part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

[Reset All Filters](#)

Vendor: All | Name: Basys3 | Board Rev: Latest

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count	Board Rev	Available IOBs	LUT Elements	FlipFlops
Basys3			digilentinc.com	1.2	xc7a35tcbg236-1	236	C.0	106	20800	41600

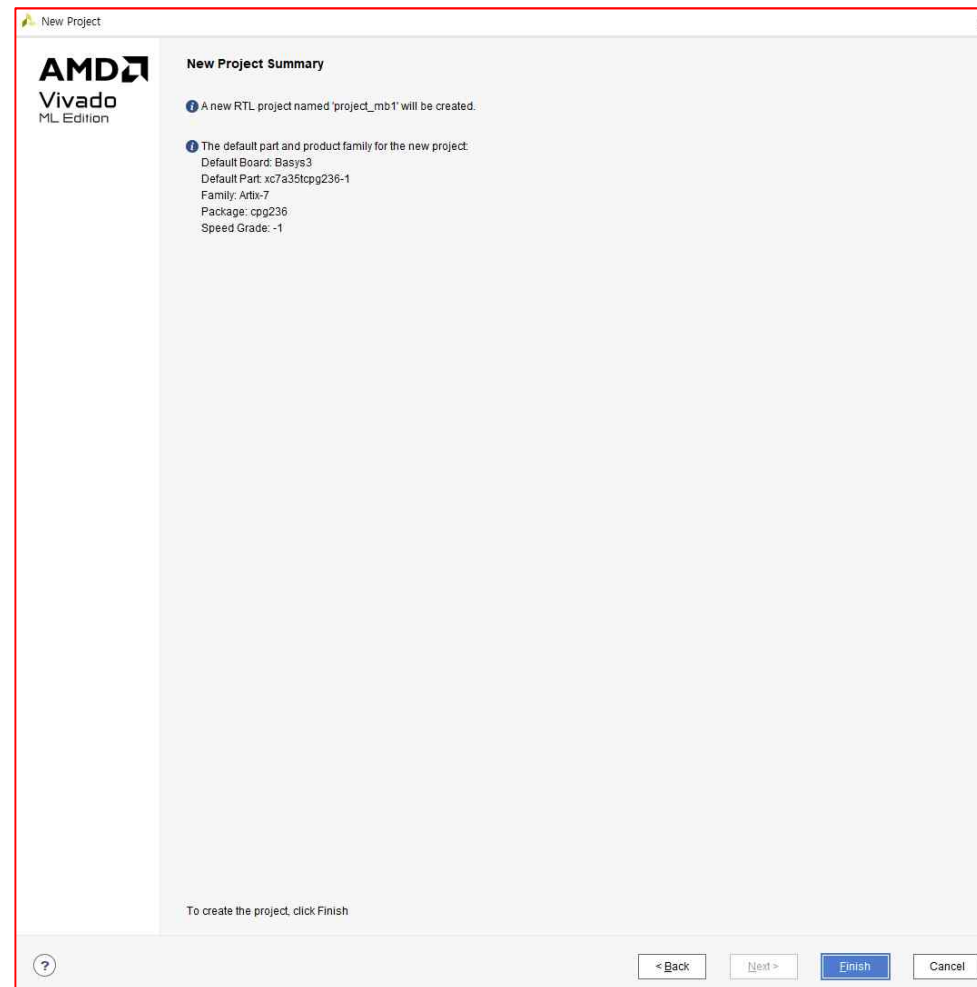
[Refresh](#) Catalog was last updated on 06/01/2023 6:42:45 AM

[? < Back](#) [Next >](#) [Enish](#) [Cancel](#)

Hello World Implementation

Microblaze

- **Hello World Implementation**
 - **New Project Summary ➔ Finish**

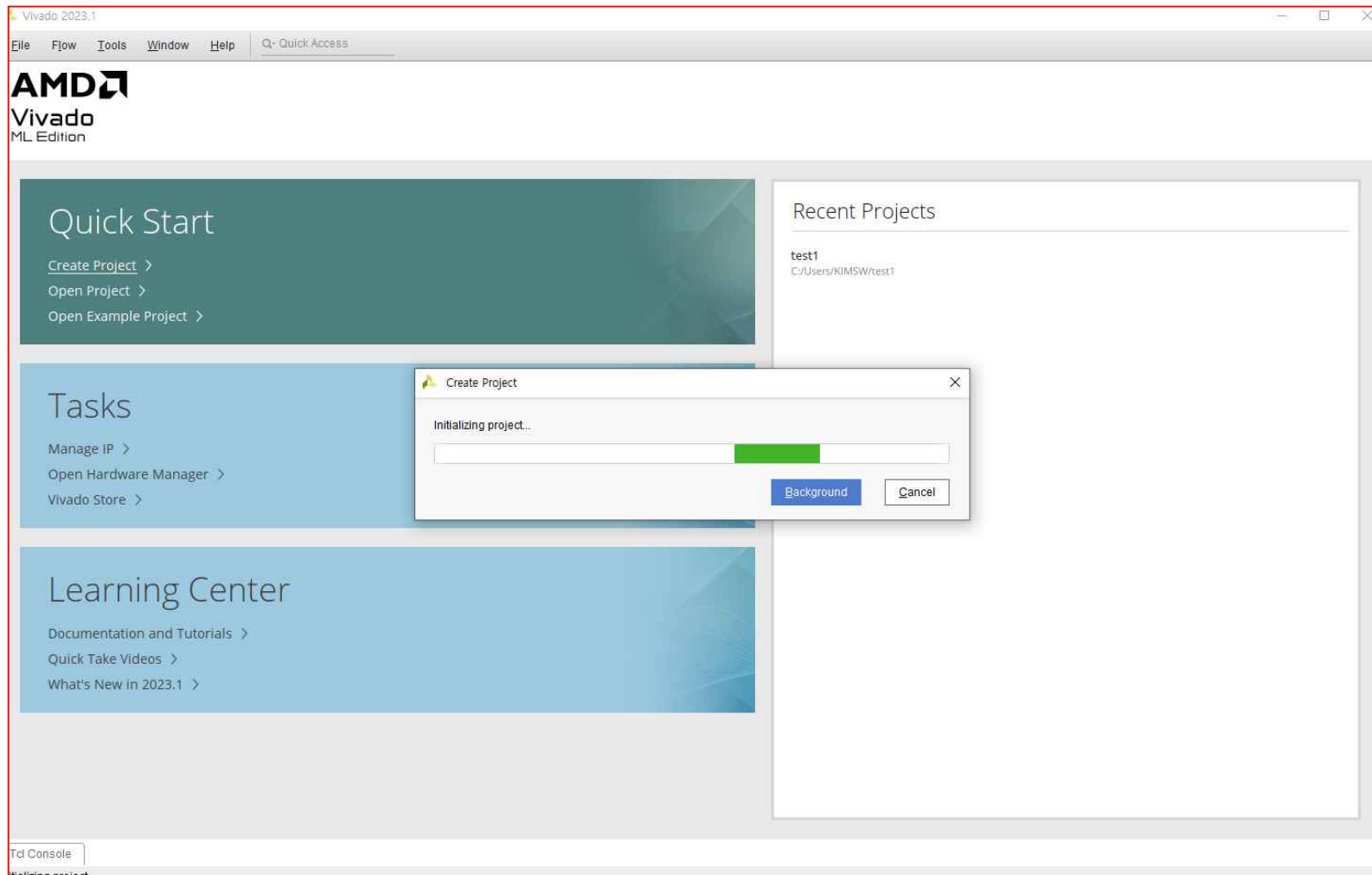


Hello World Implementation

Microblaze

➤ Hello World Implementation

- Create Project 완료



Hello World Implementation

Microblaze

➤ Hello World Implementation

- Create Project 완료 ➔ **PROJECT MANAGER**

The screenshot shows the Vivado PROJECT MANAGER interface for a project named 'project_mb1'. The left sidebar contains a 'How Navigator' with sections for PROJECT MANAGER, IP INTEGRATOR, SIMULATION, RTL ANALYSIS, SYNTHESIS, IMPLEMENTATION, and PROGRAM AND DEBUG. The main area is divided into three panes: Sources, Properties, and Project Summary. The Project Summary pane is active, showing details for the project and the selected board part (Basys3). The bottom status bar displays a table of design runs.

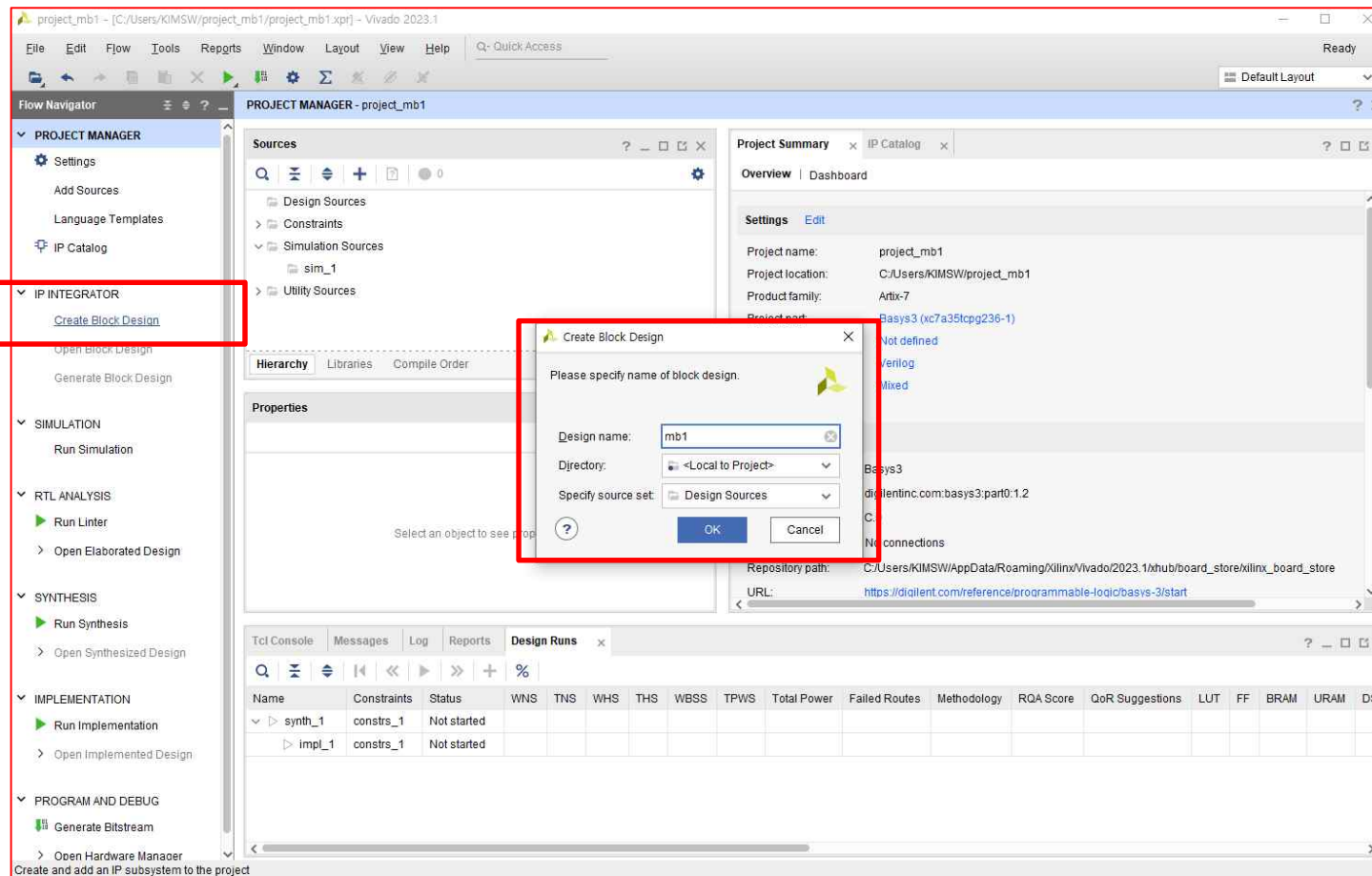
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	QoR Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strategy	Report Strategy	Part	Description
synth_1	constrs_1	Not started																			Vivado Synthesis Defaults (Vivado Synthesis 2023)	Vivado Synthesis Default Reports (Vivado Synthesis 2023)	xc7a35kpg236-1	Vivado Synth
impl_1	constrs_1	Not started																			Vivado Implementation Defaults (Vivado Implementation 2023)	Vivado Implementation Default Reports (Vivado Implementation 2023)	xc7a35kpg236-1	Default sett

Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

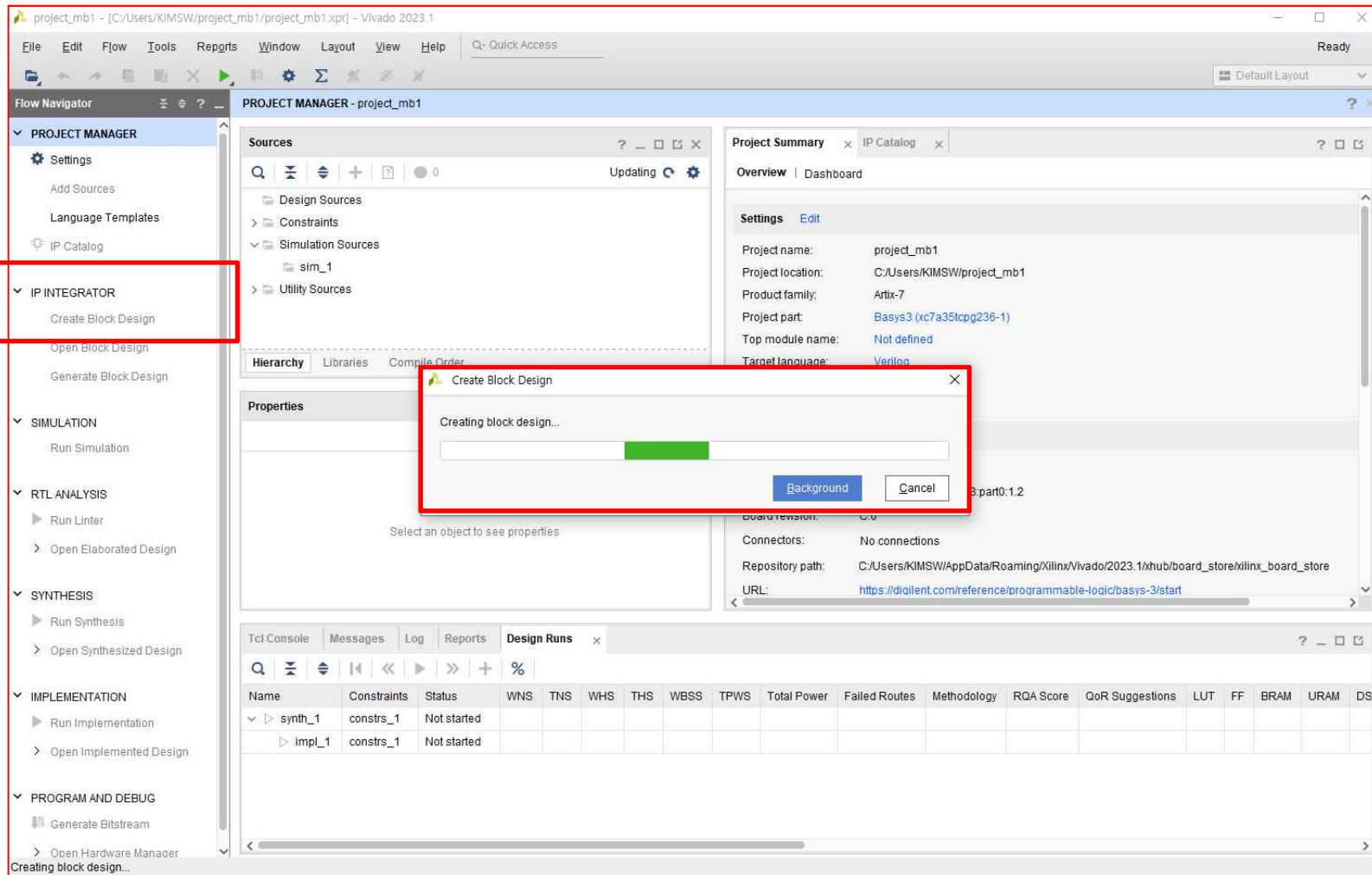
- PROJECT MANAGER → IP INTEGRATOR → Create Block Design
- Microblaze : 사용자가 원하는 IP Block 을 선택 , 속성 지정 및 각 Block 들을 연결해서 사용
- [Design Name(ex. Microblaze_1)] 입력 → OK



Hello World Implementation

Microblaze

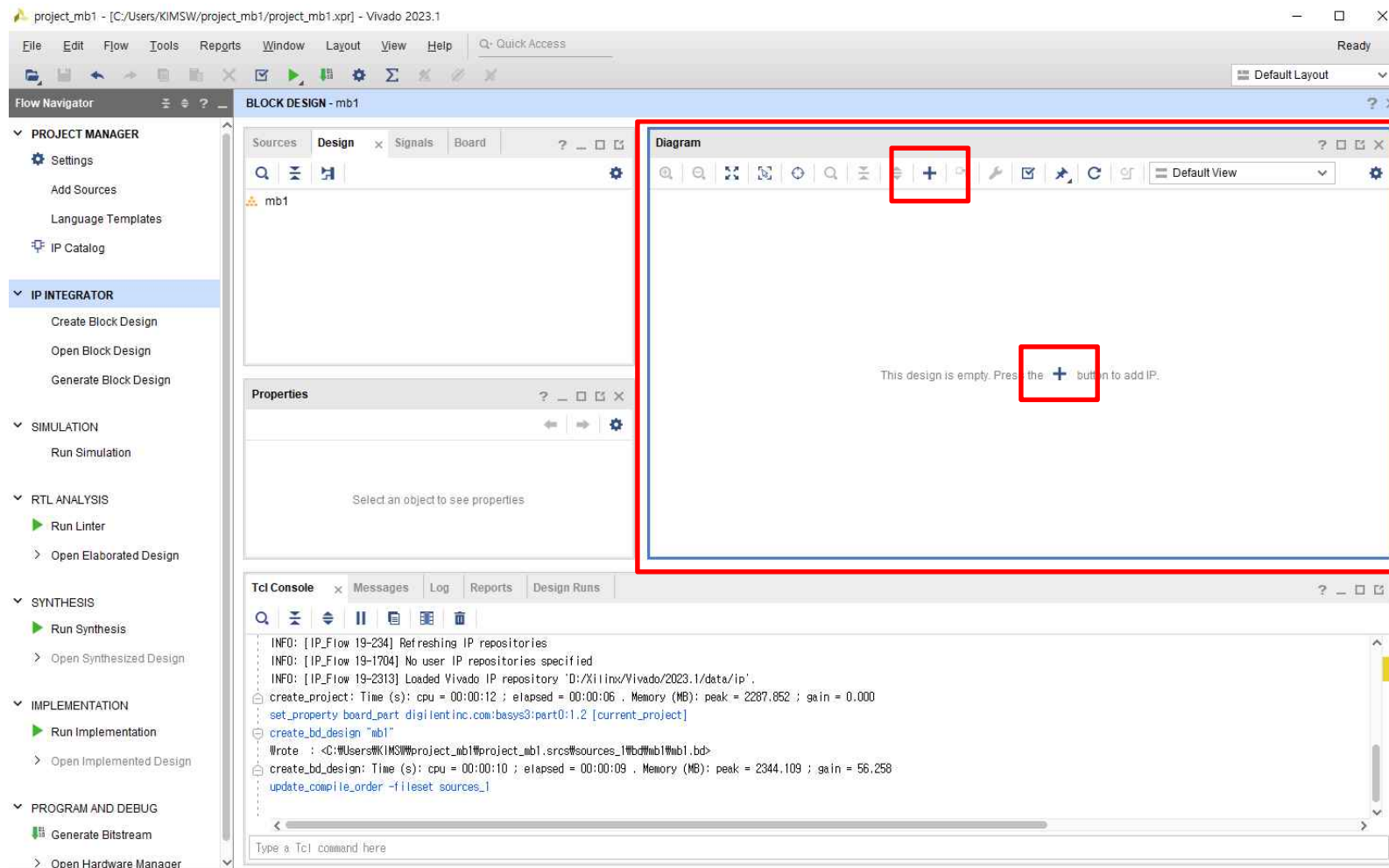
- Hello World Implementation → Microblaze 생성
 - IP INTEGRATOR → Create Block Design



Hello World Implementation

Microblaze

- **Hello World Implementation** → **Microblaze** 생성
 - **Diagram Window** → **Add IP** → “+” 아이콘 클릭

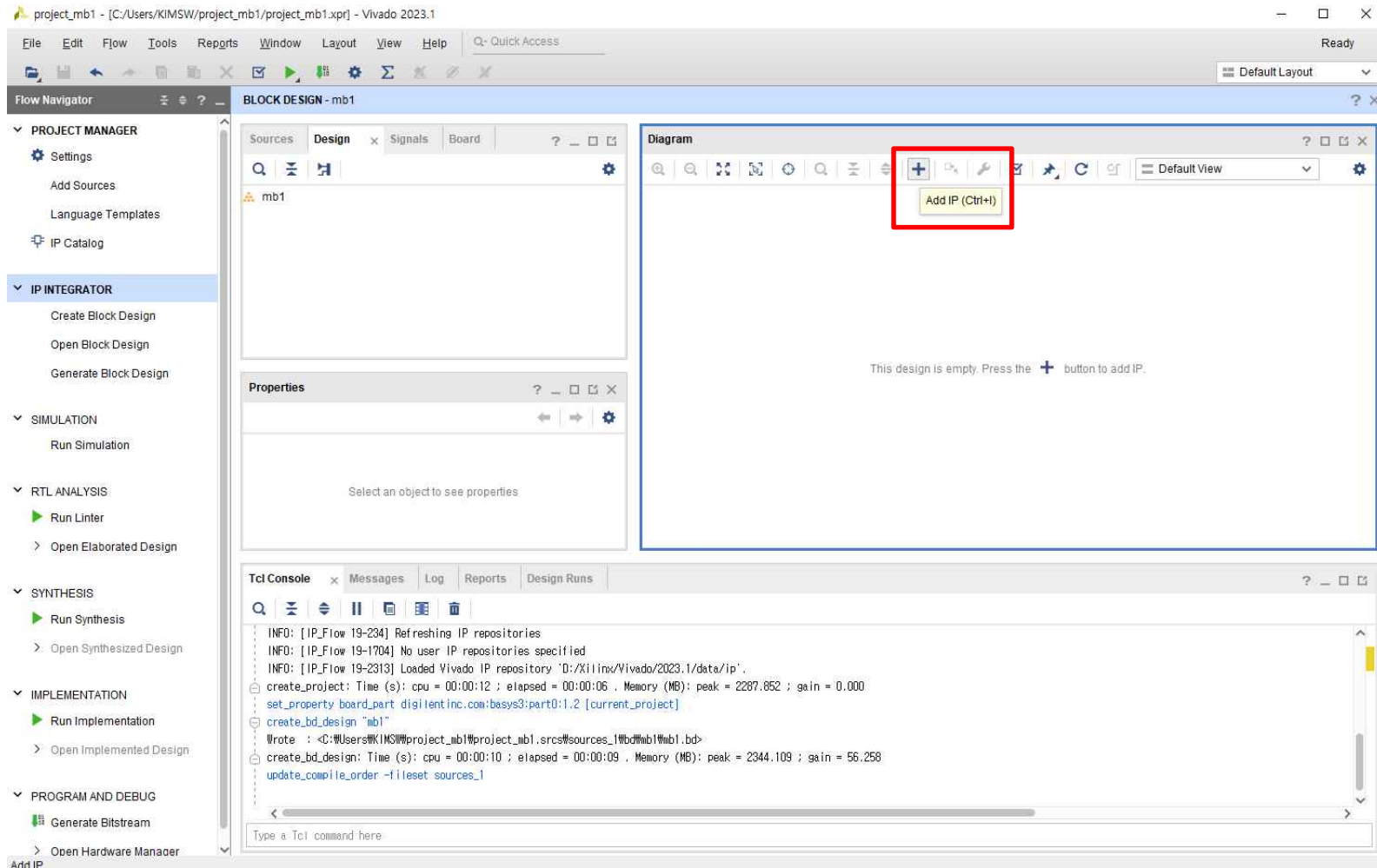


Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

- Diagram Window → Add IP → “+” 아이콘 클릭 → “MicroBlaze” 검색 및 IP [MicroBlaze] 추가(1)

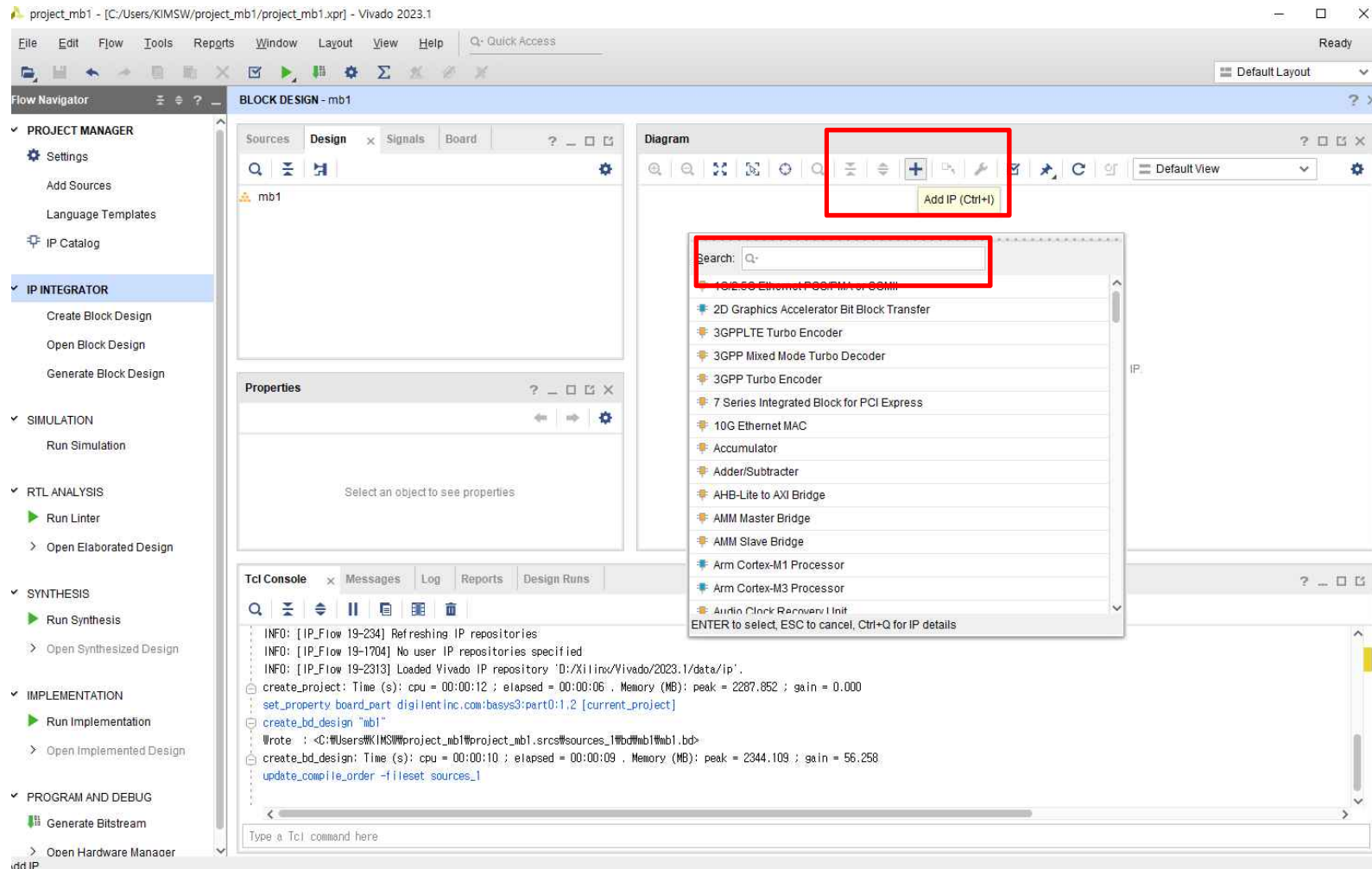


Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

- Diagram Window → Add IP → “+” 아이콘 클릭 → “MicroBlaze” 검색 및 IP [MicroBlaze] 추가(2)

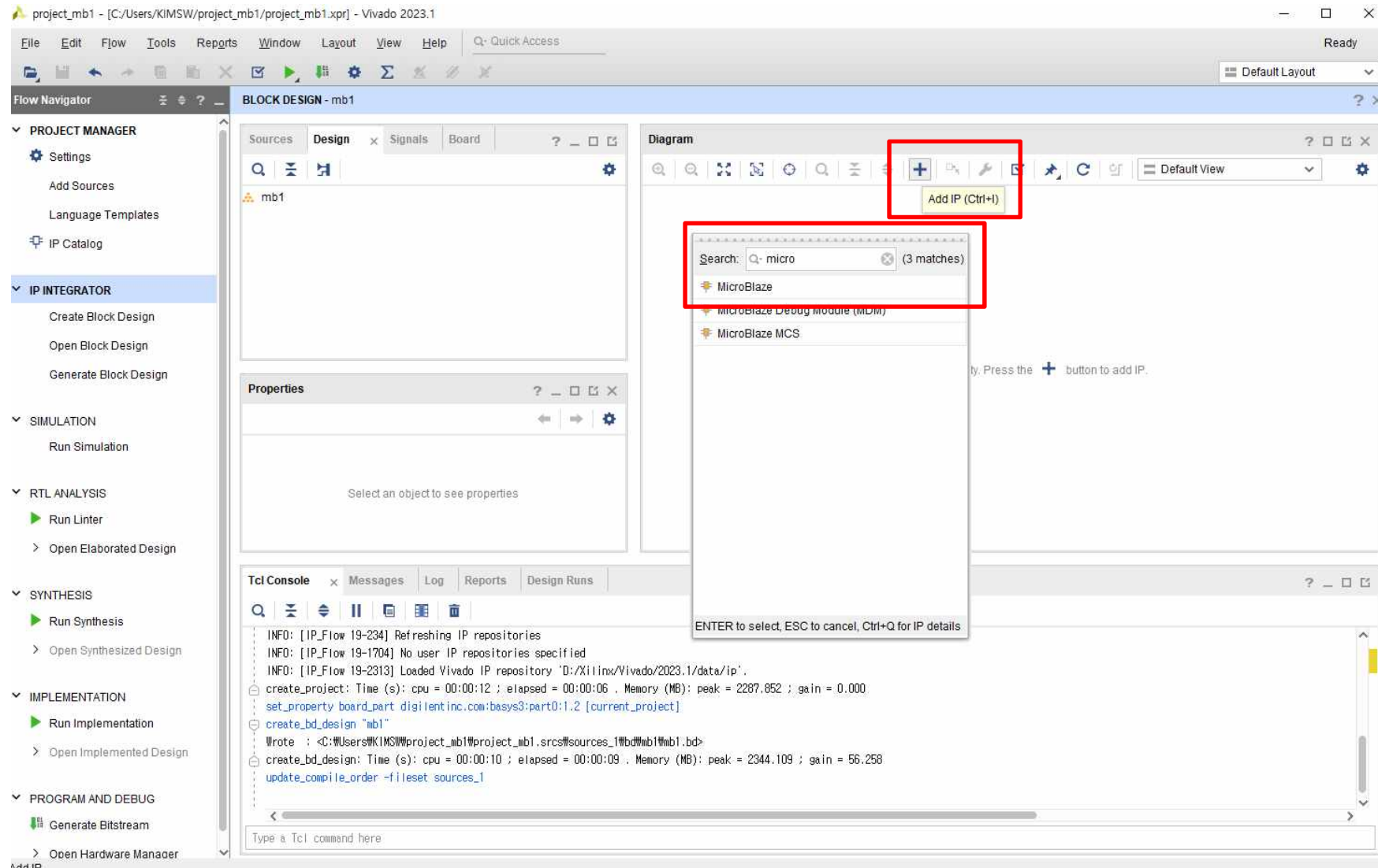


Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

- Diagram Window → Add IP → “+” 아이콘 클릭 → “MicroBlaze” 검색 및 IP [MicroBlaze] 추가(3)

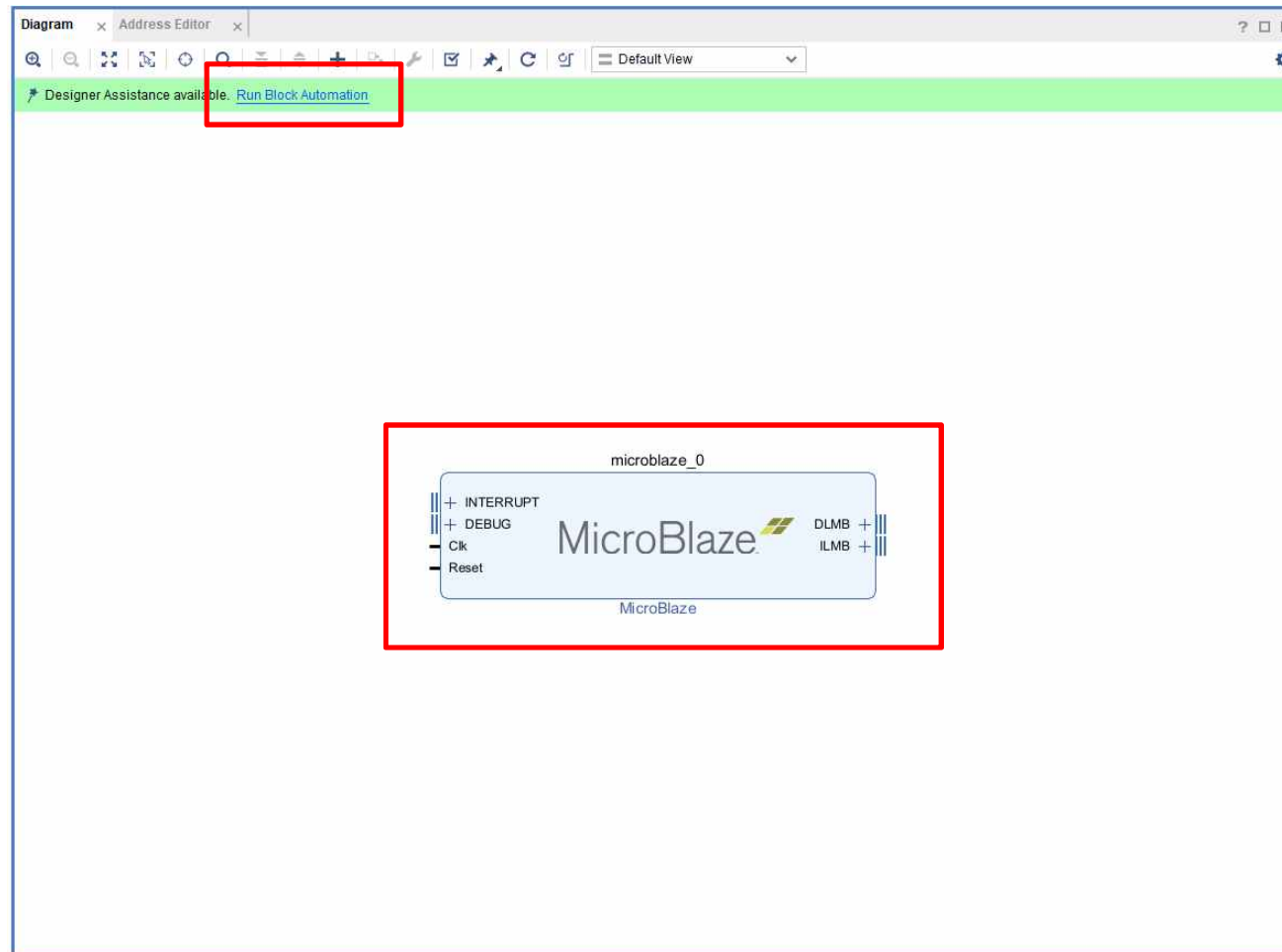


Hello World Implementation

Microblaze

➤ **Hello World Implementation** → Microblaze 생성

- **Diagram Window** → **Add IP** → “+” 아이콘 클릭 → **“MicroBlaze” 검색 및 IP [MicroBlaze] 추가(4)**
- **Run Block Automation** → **Options**



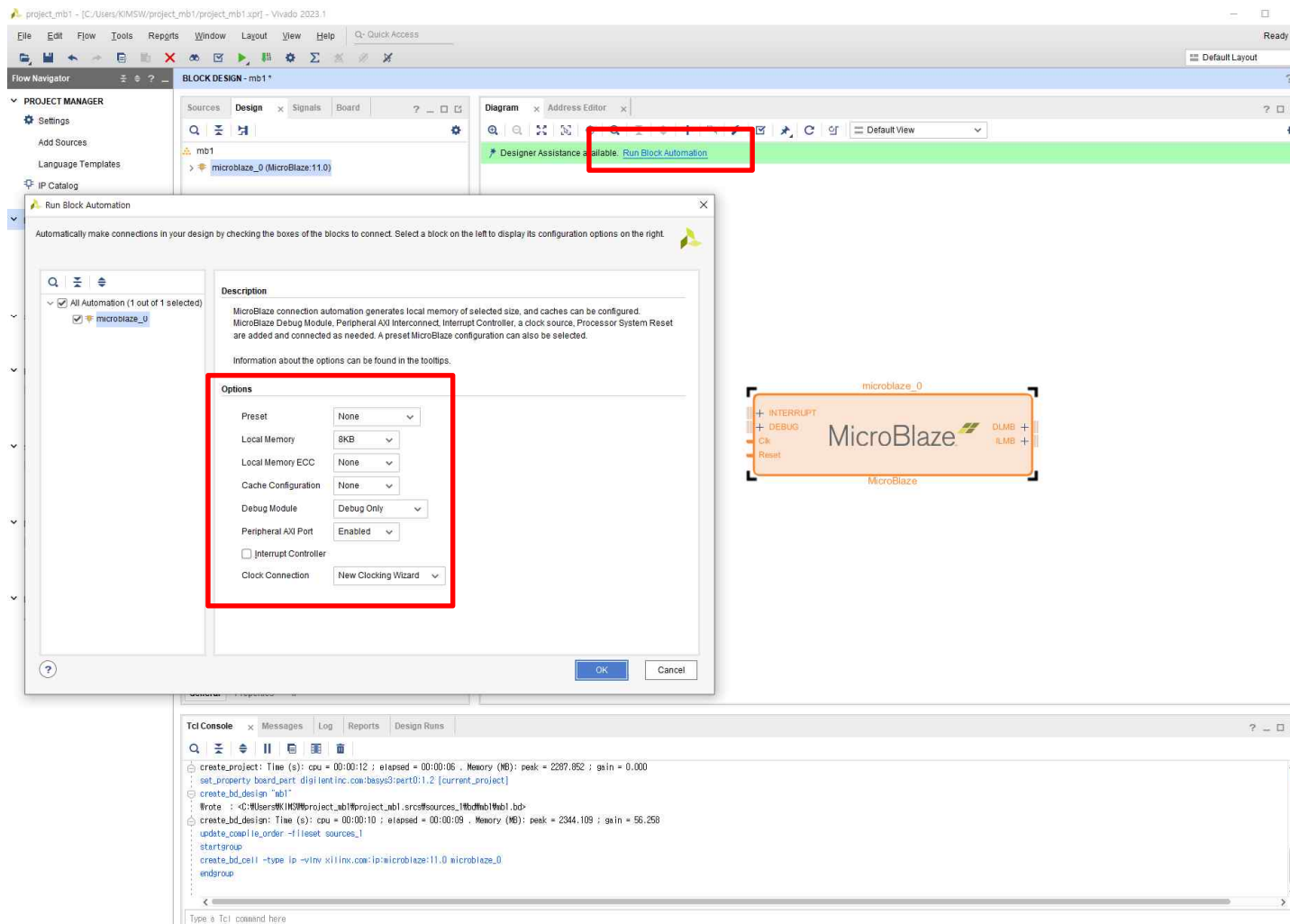
Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

■ Run Block Automation → Options

[Local Memory : Code Memory] 설정 → OK



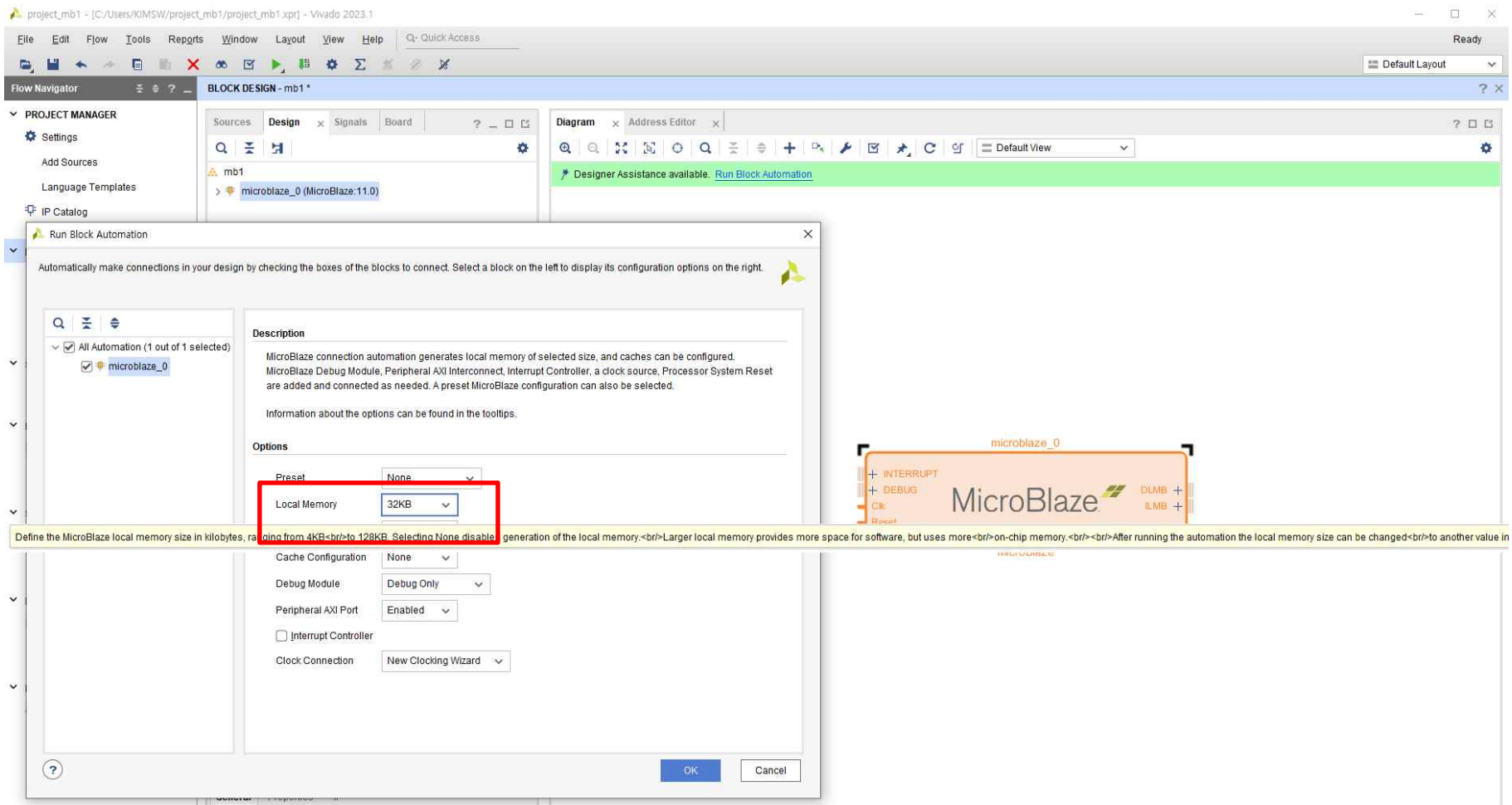
Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

■ Run Block Automation → Options

☞ [Local Memory : Code Memory] 설정 → OK

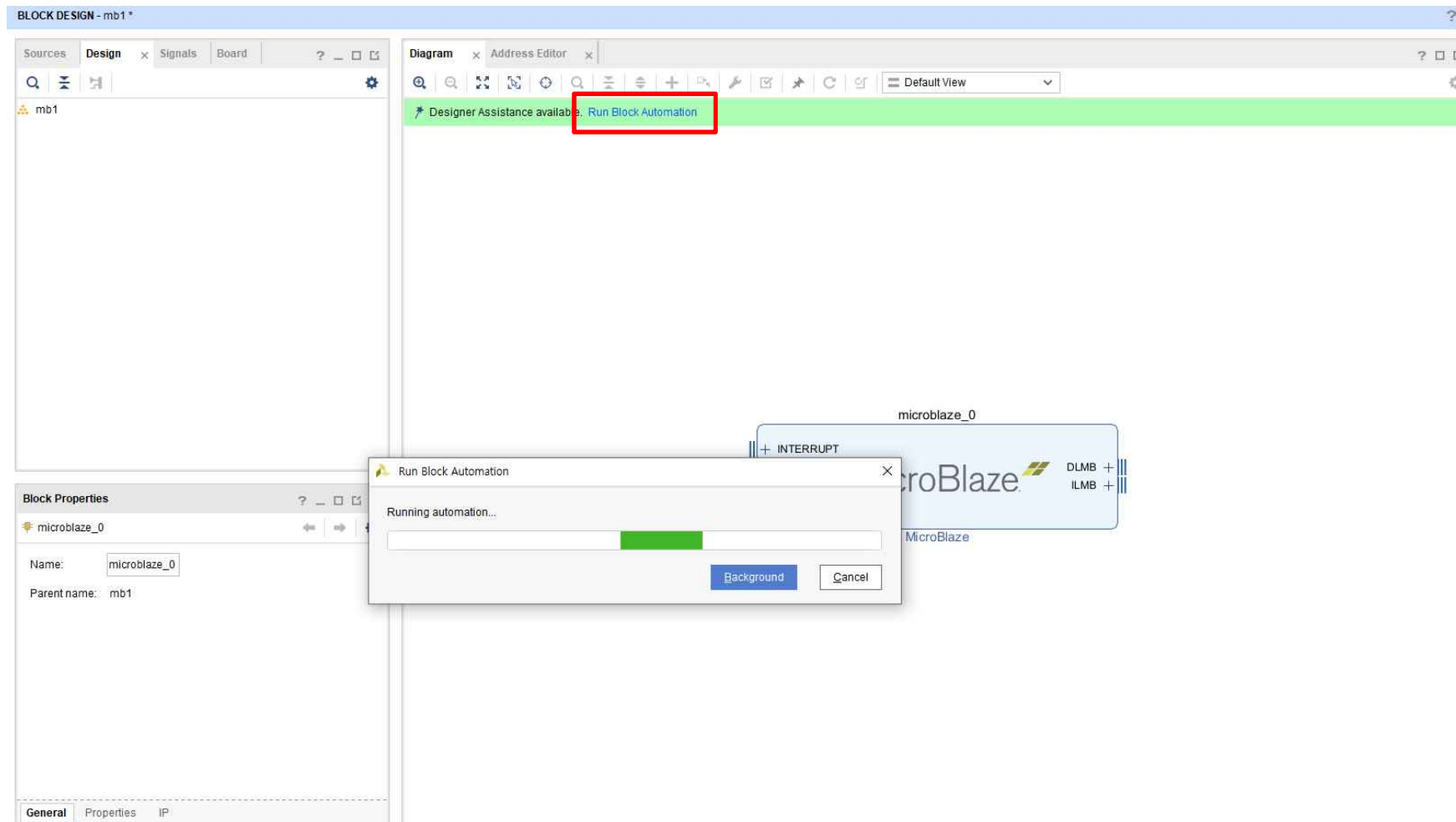


Hello World Implementation

Microblaze

➤ Hello World Implementation

- **Run Connection Automation** ➔ Block 자동 연결
- IP 추가 및 수정 후 ➔ **[Run Block Automation] & [Run Connection Automation]** 기능 적용

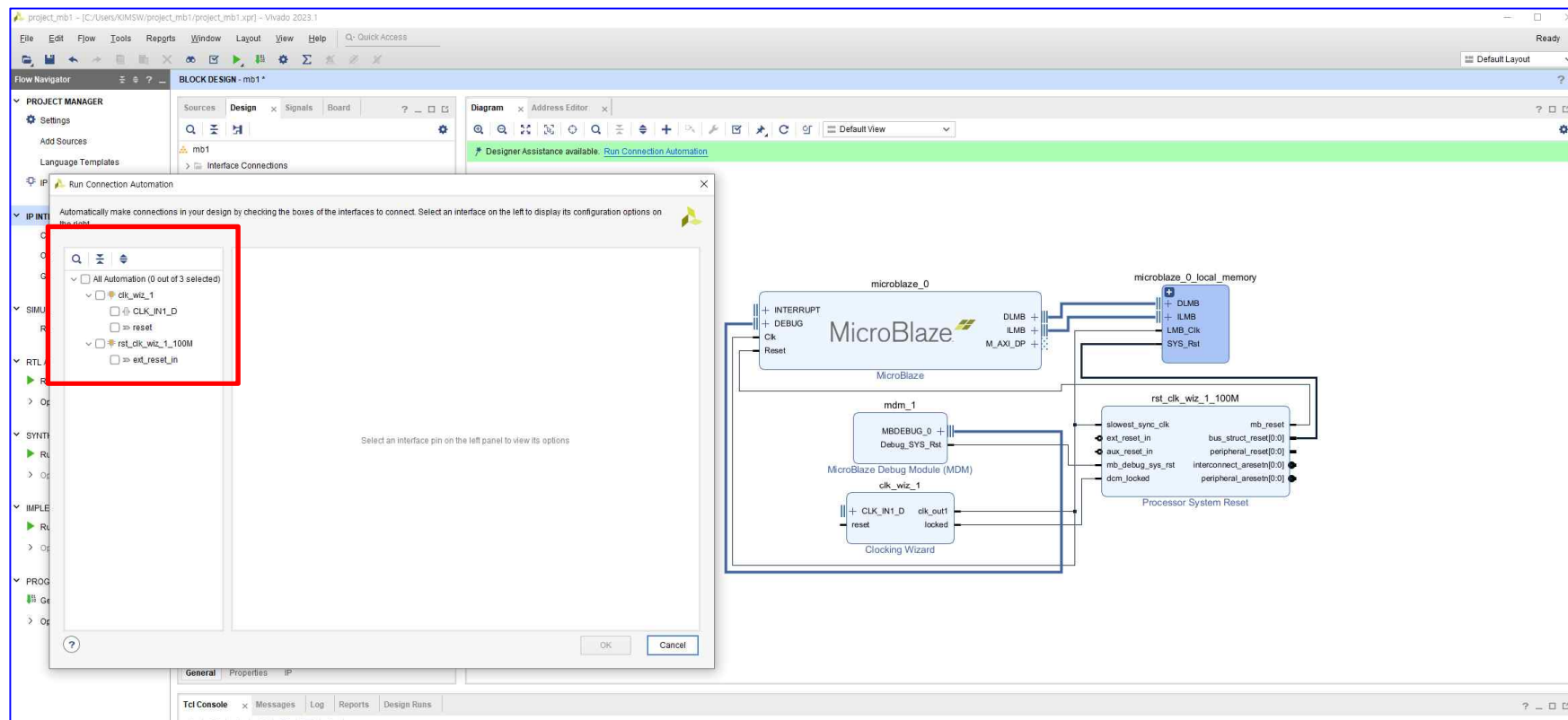


Hello World Implementation

Microblaze

➤ Hello World Implementation

- Run Connection Automation ➔ Block 자동 연결
- [All Automation ➔ OK] 생성된 Block 자동 연결

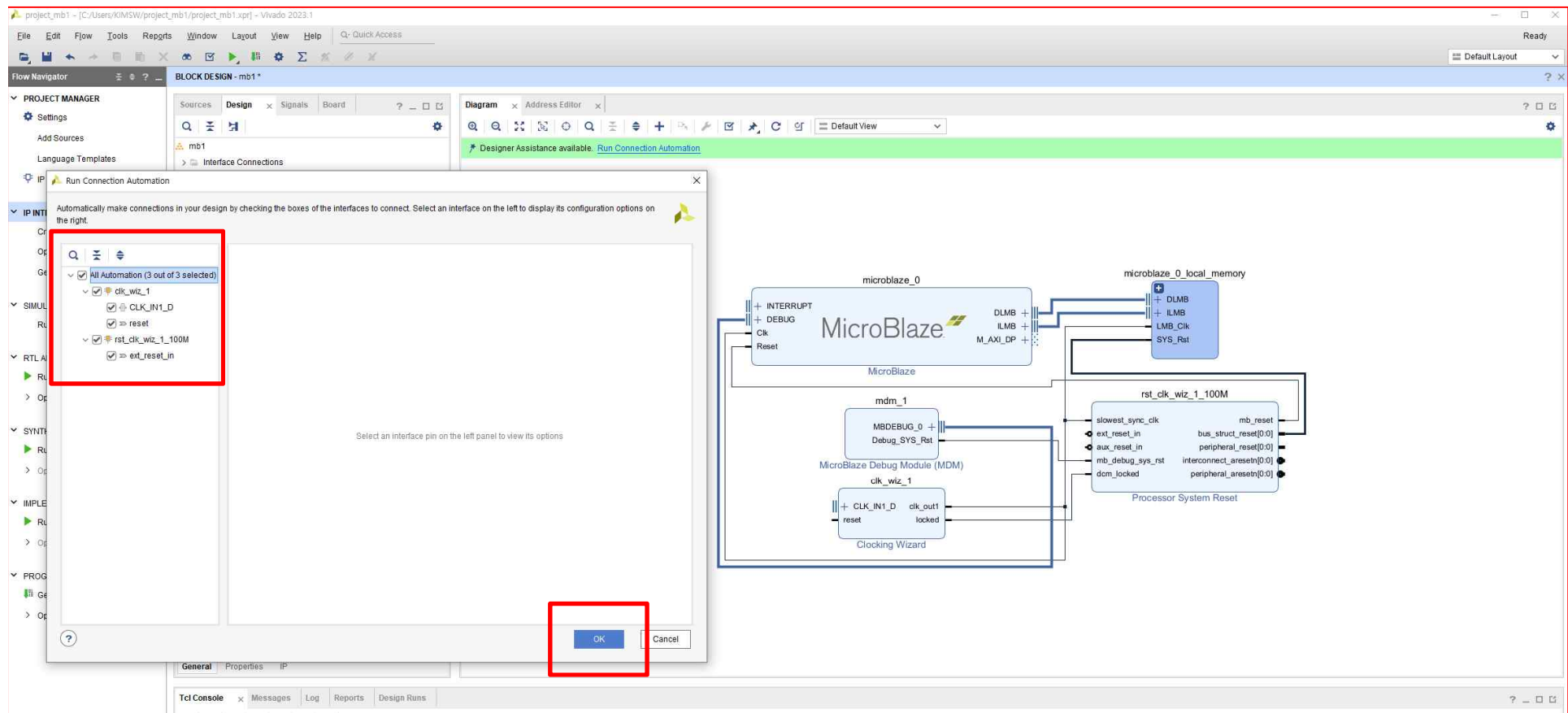


Hello World Implementation

Microblaze

➤ Hello World Implementation

- Run Connection Automation ➔ Block 자동 연결
- **[All Automation ➔ OK]** 생성된 Block 자동 연결

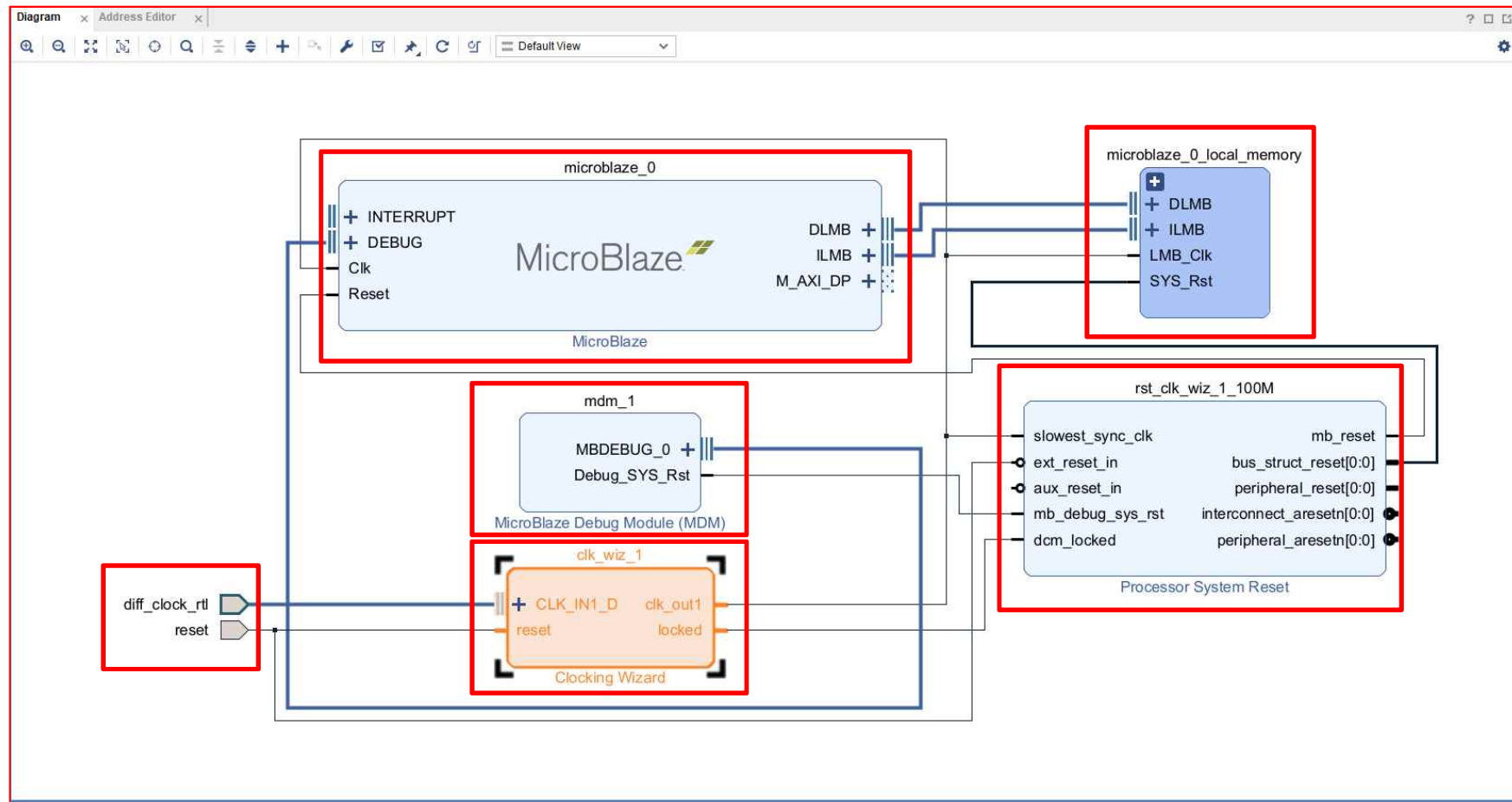


Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

- [MicroBlaze Core // Clock // Reset // Local Memory // Debug Module] 생성

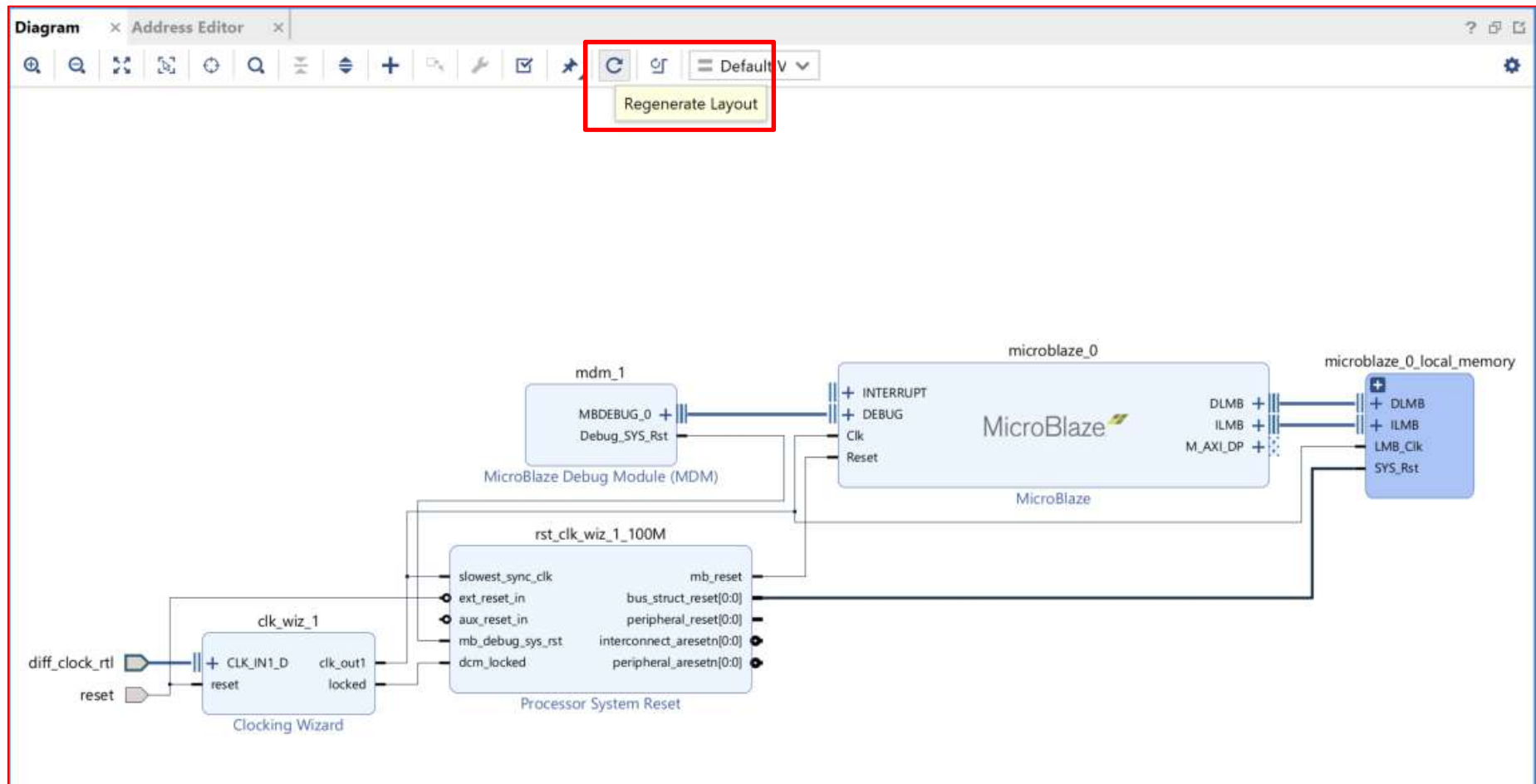


Hello World Implementation

Microblaze

➤ Hello World Implementation

- 배치 및 정리 ➔ *Regenerate Layout* 클릭

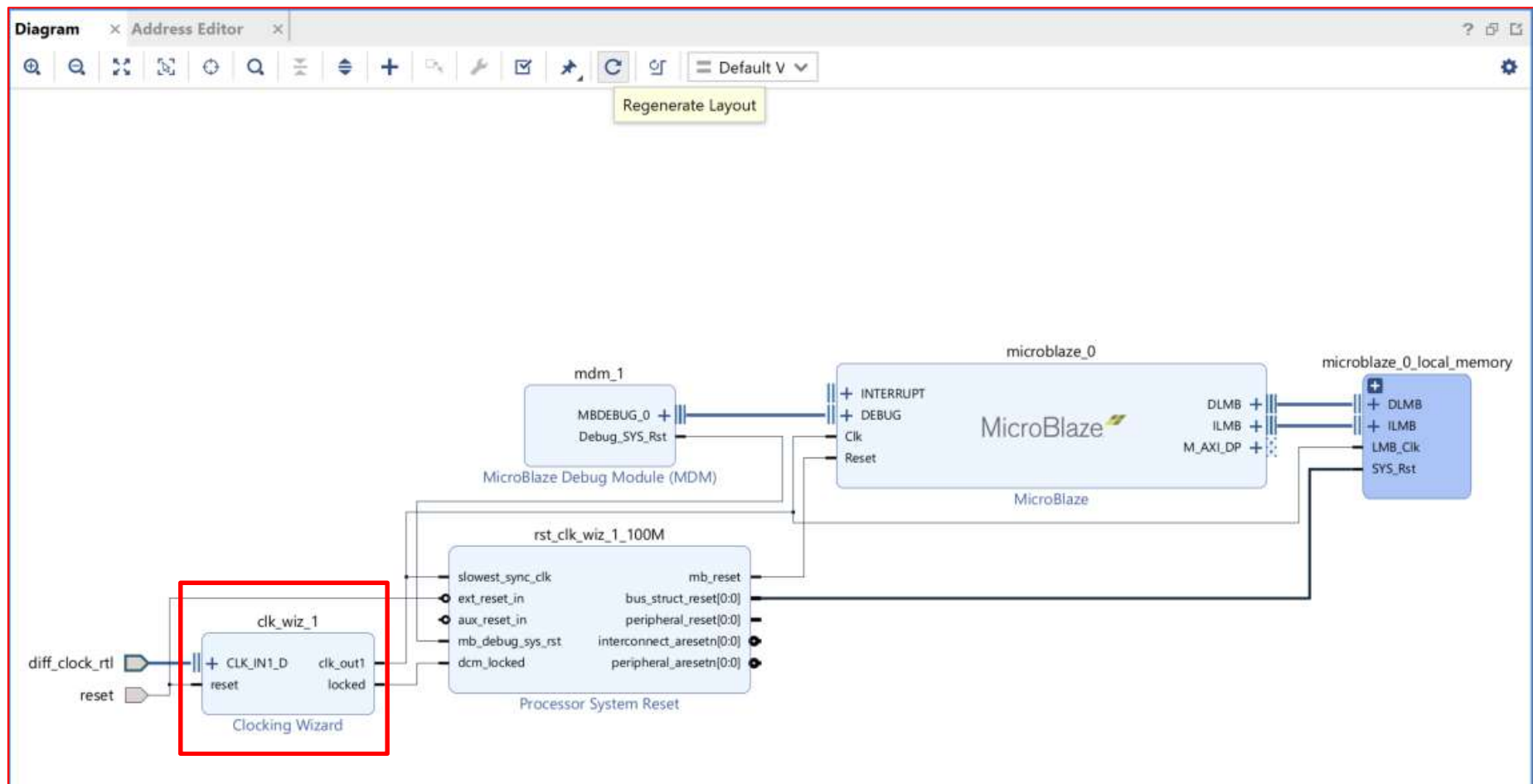


Hello World Implementation

Microblaze

➤ Hello World Implementation

- Basys3(Artix 7) HW를 위한 속성 변경
 - *clk_wiz_1* 속성 변경 ➔ Double Click



Hello World Implementation

Microblaze

➤ Hello World Implementation → Microblaze 생성

- **clk_wiz_1 Block**을 선택 후 Double 클릭 → 속성 변경
- **Board** 탭 → **CLK_IN1 : sys_clock**
- **Clocking Options** 탭 → **clk_in1 : Single ended clock** [← sys_clock 설정시 자동 설정]
- **Output Clocks** 탭 → **Reset Type : Active Low (or Active High)**

The image displays three screenshots of the Xilinx Clocking Wizard (6.0) configuration interface, illustrating the steps to configure the clocking for a Microblaze Hello World implementation.

Top Screenshot: Board Tab

The **Board** tab is selected. The **IP Interface** section shows the following configuration:

IP Interface	Board Interface
CLK_IN1	sys_clock
CLK_IN2	Custom
EXT_RESET_IN	reset

The **Clear Board Parameters** button is visible below the table.

Bottom Left Screenshot: Clocking Options Tab

The **Clocking Options** tab is selected. The **Primitive** section shows **MMCM** selected. The **Clocking Features** section shows **Frequency Synthesis**, **Phase Alignment**, and **Dynamic Reconfig** checked. The **Dynamic Reconfig Interface** section shows **AXI4/AXI4-Lite** selected. The **Input Clock Information** table is highlighted with a red box:

Input Clock	Port Name	Input Frequency(MHz)	Jitter Options	Input Jitter	Source
Primary	clk_in1	100.000	10.000 - 800.000	UI	Single ended clock capable ps
Secondary	clk_in2	100.000	80.000 - 120.000	0.010	Single ended clock capable ps

Bottom Right Screenshot: Output Clocks Tab

The **Output Clocks** tab is selected. The **Output Clock** table is highlighted with a red box:

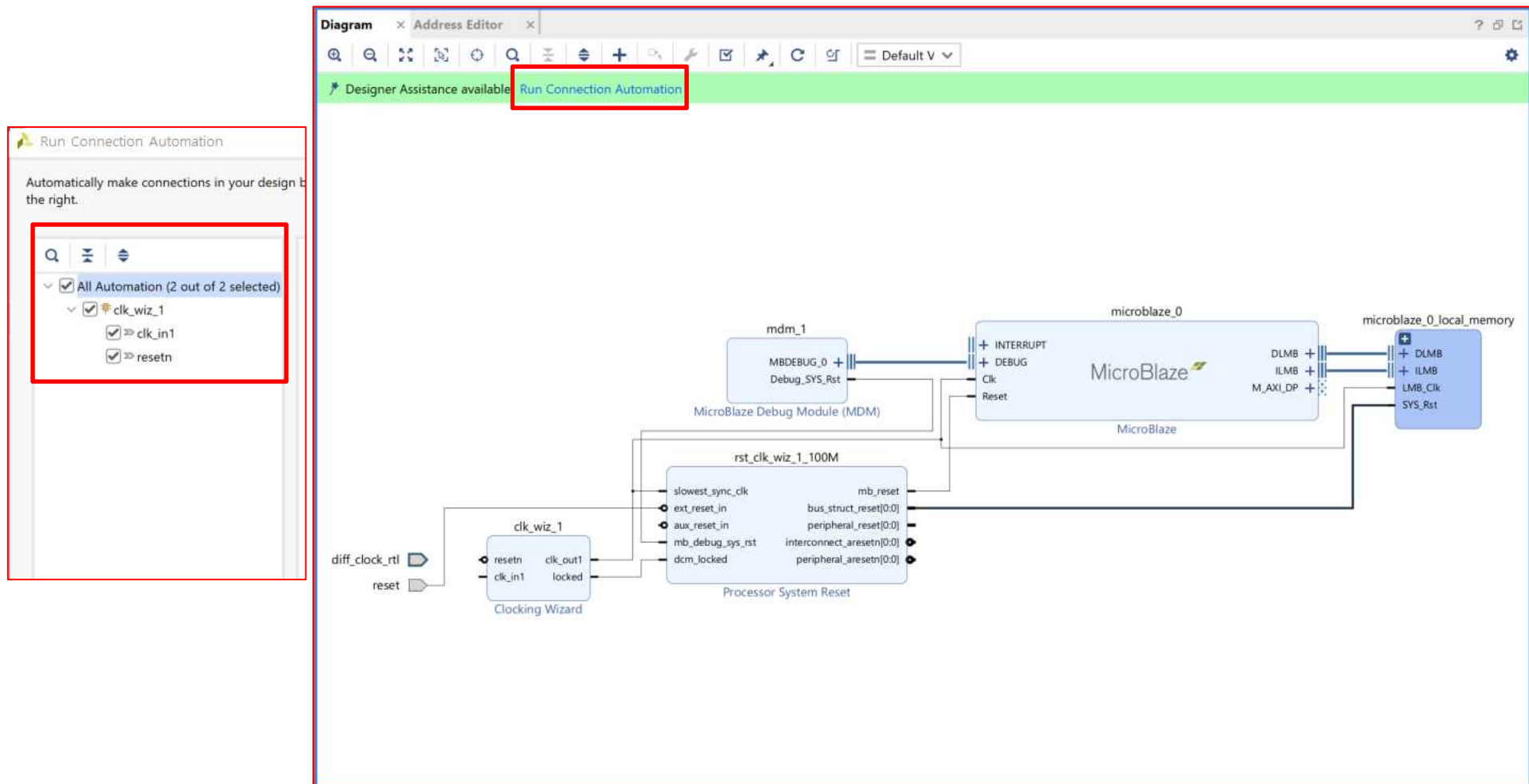
Output Clock	Port Name	Output Freq (MHz)	Phase (degrees)	Duty Cycle (%)	Drives
clk_out1	clk_out1	100.000	0.000	50.000	BUFG
clk_out2	clk_out2	100.000	0.000	50.000	BUFG
clk_out3	clk_out3	100.000	0.000	50.000	BUFG
clk_out4	clk_out4	100.000	0.000	50.000	BUFG
clk_out5	clk_out5	100.000	0.000	50.000	BUFG
clk_out6	clk_out6	100.000	0.000	50.000	BUFG
clk_out7	clk_out7	100.000	0.000	50.000	BUFG

The **Reset Type** section shows **Active Low** selected. The **Enable Optional Inputs / Outputs for MMCM/PLL** section shows **reset** checked.

Hello World Implementation

Microblaze

- **Hello World Implementation** → **Microblaze** 생성
 - **clk_wiz_1 Block**을 선택 후 **Double** 클릭 → 속성 변경
 - 속성 변경 완료 후 → **Run Connection Automation**

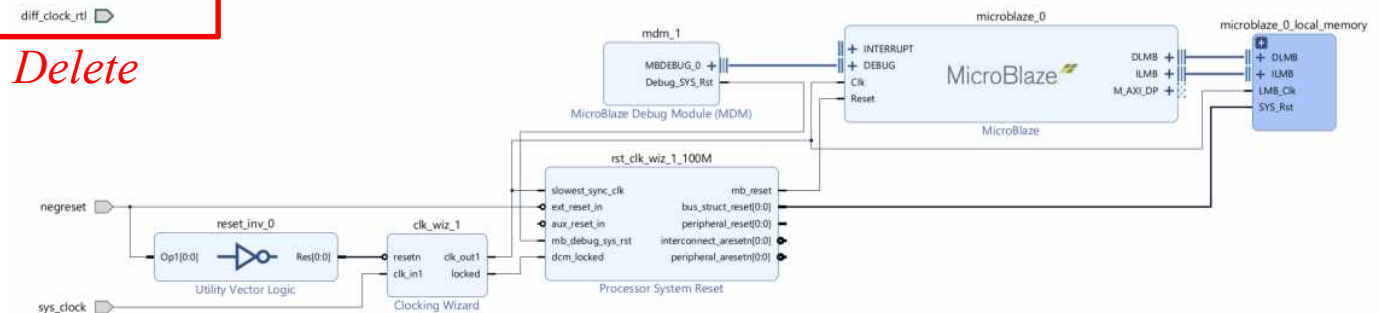


Hello World Implementation

Microblaze

➤ Hello World Implementation

- 포트와 블록 정리 및 이름 변경
- *reset_0* : *negreset* 으로 변경 (해당 핀 선택) → *External Port Properties* 창에서 *Name* : *negreset* 으로 변경)
- [*diff_clock_rtl* 포트, *reset_inv_0*] *Block* 제거 (선택 후 Delete 키로 제거)
- 배치 및 정리 → *Regenerate Layout* 클릭

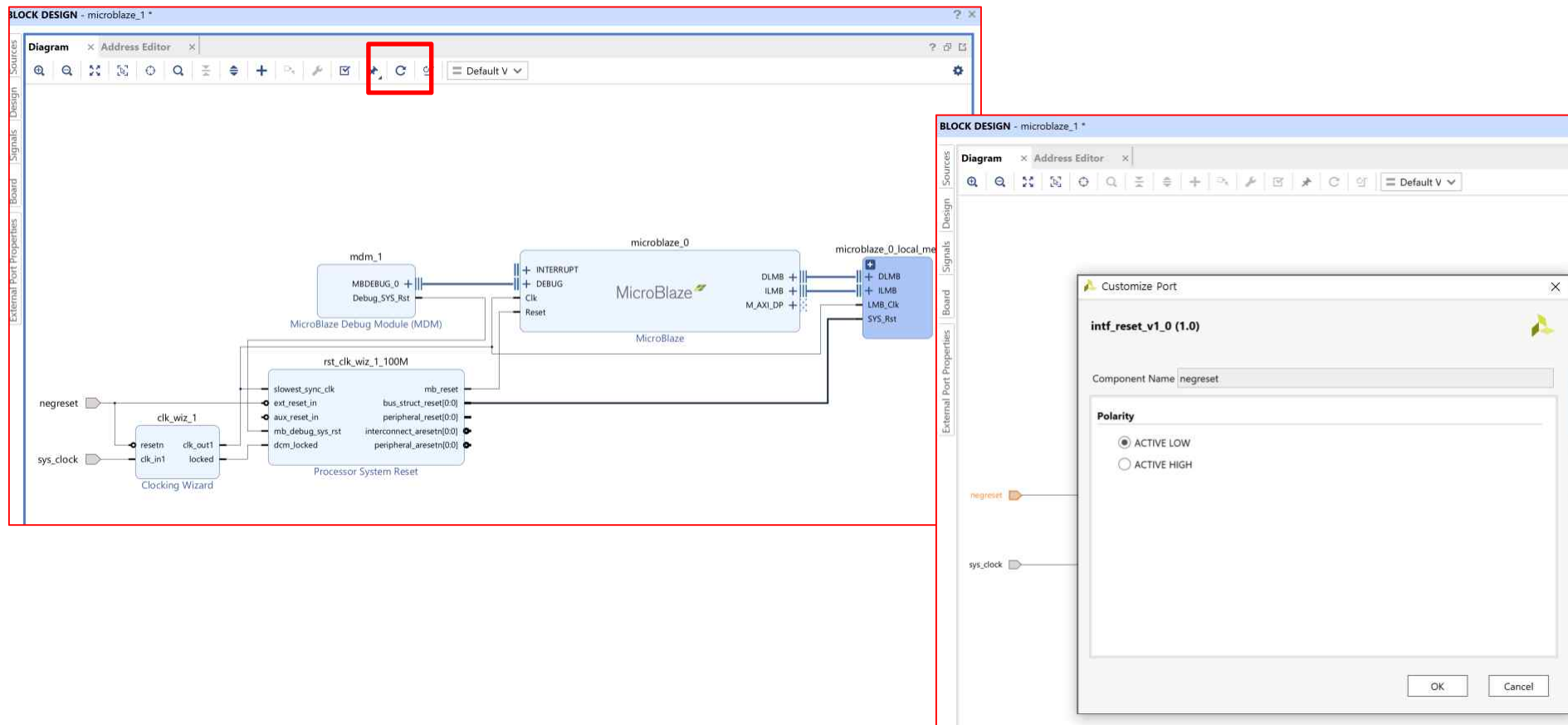


Hello World Implementation

Microblaze

➤ Hello World Implementation

- 포트와 블록 정리 및 이름 변경
- *reset_0 : negreset* 으로 변경 (해당 핀을 선택) → *External Port Properties* 창에서 *Name : negreset* 으로 변경)
- [*diff_clock_rtl* 포트, *reset_inv_0*] block 제거 (선택 후 *Delete* 키로 제거)
- 배치 및 정리 → Regenerate Layout 클릭 → [*negreset* → *Active Low* 변경]

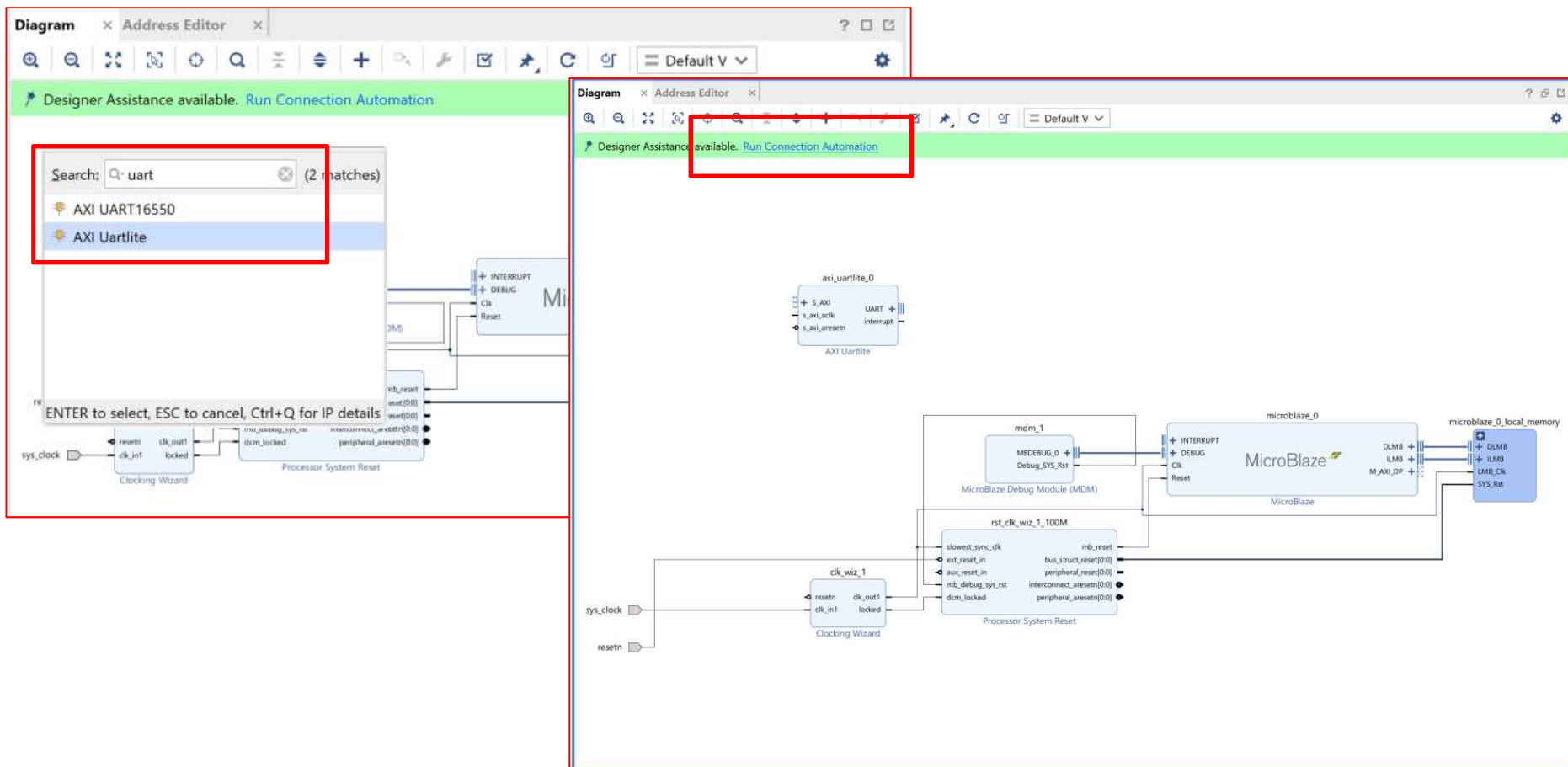


Hello World Implementation

Microblaze

➤ Hello World Implementation → UART IP 추가

- Add IP 아이콘 (+)을 클릭 → **UART Block**을 추가
- UART 입력 → **[AXI Uartlite]** → Double Click → “Run Connection Automation”을 클릭해서 연결
- Regenerate Layout

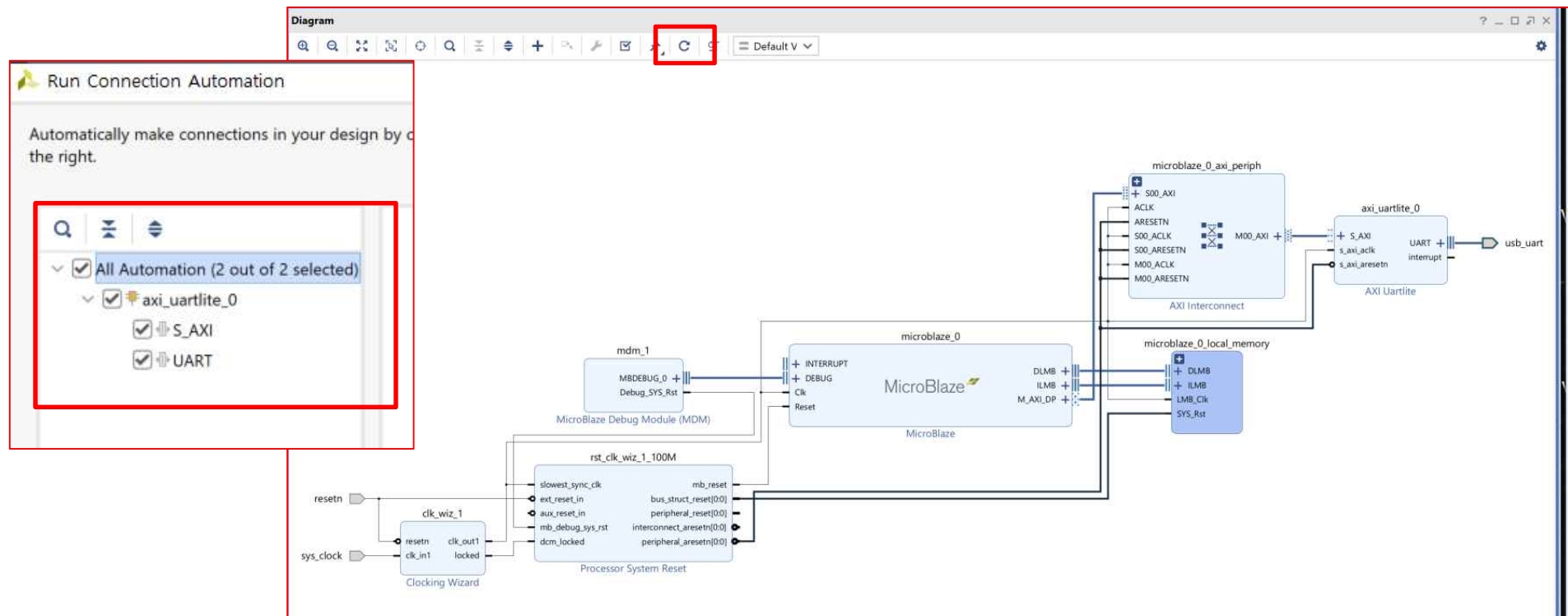


Hello World Implementation

Microblaze

➤ Hello World Implementation → UART IP 추가

- Add IP 아이콘 (+)을 클릭 → UART Block을 추가
- UART 입력 → [AXI Uartlite] → Double Click → “Run Connection Automation”을 클릭해서 연결

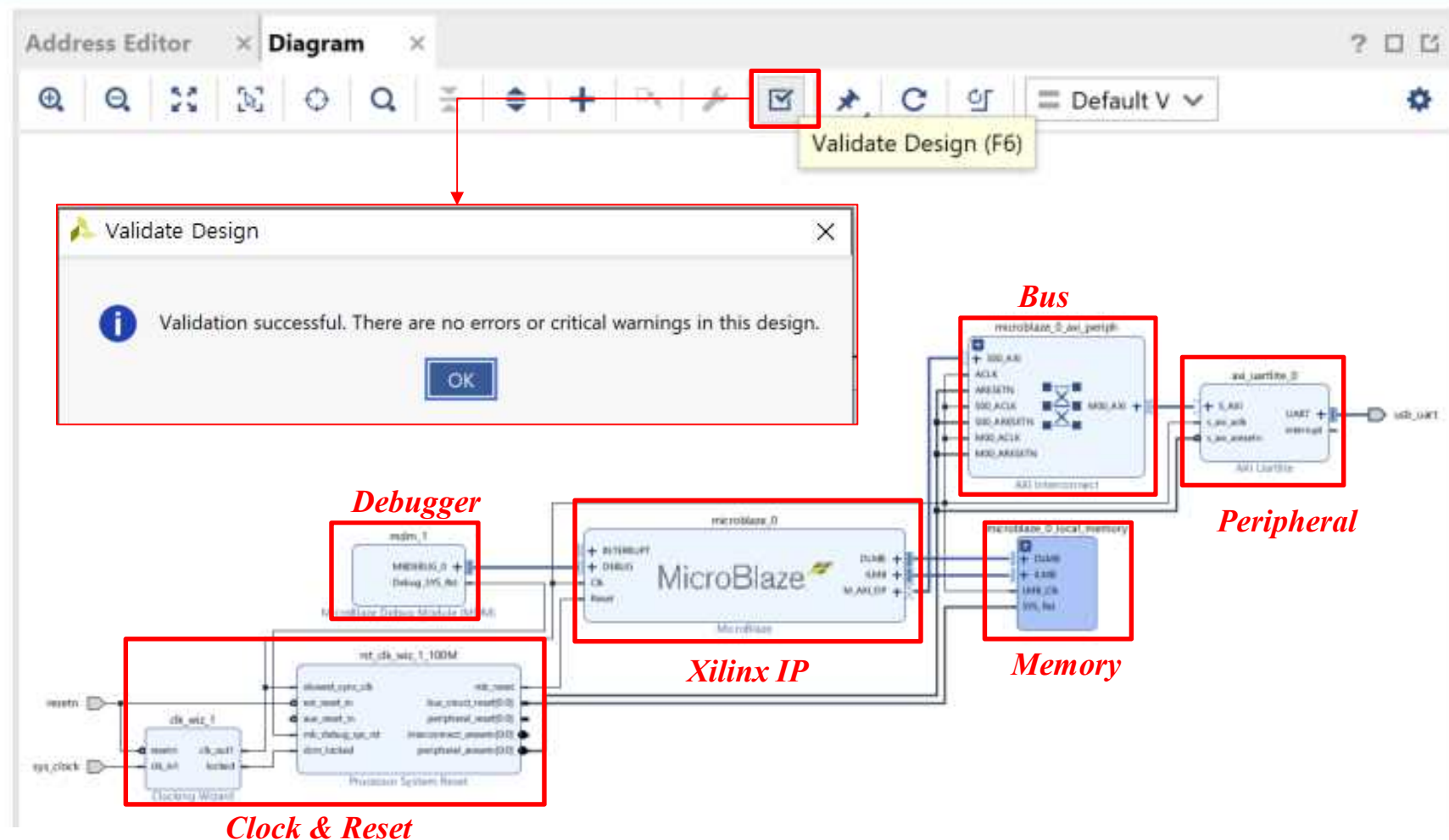


Hello World Implementation

Microblaze

➤ Hello World Implementation

- IP Block ➔ 코드 변환
- 유효성 검사 ➔ **Validate Design** 클릭 ➔ **Validation Success**

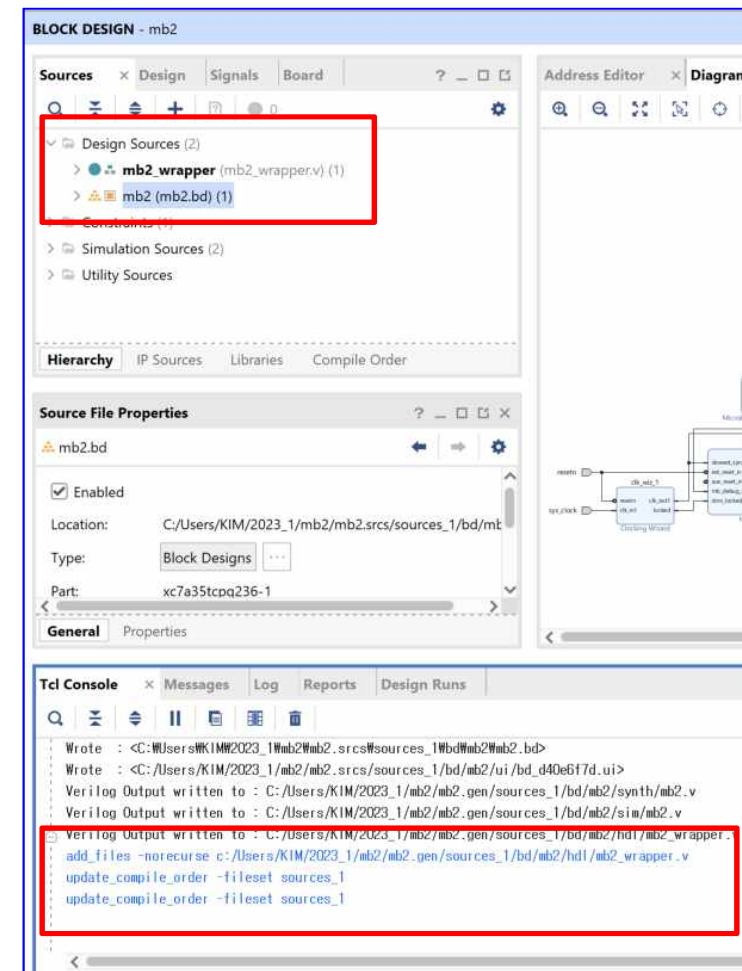
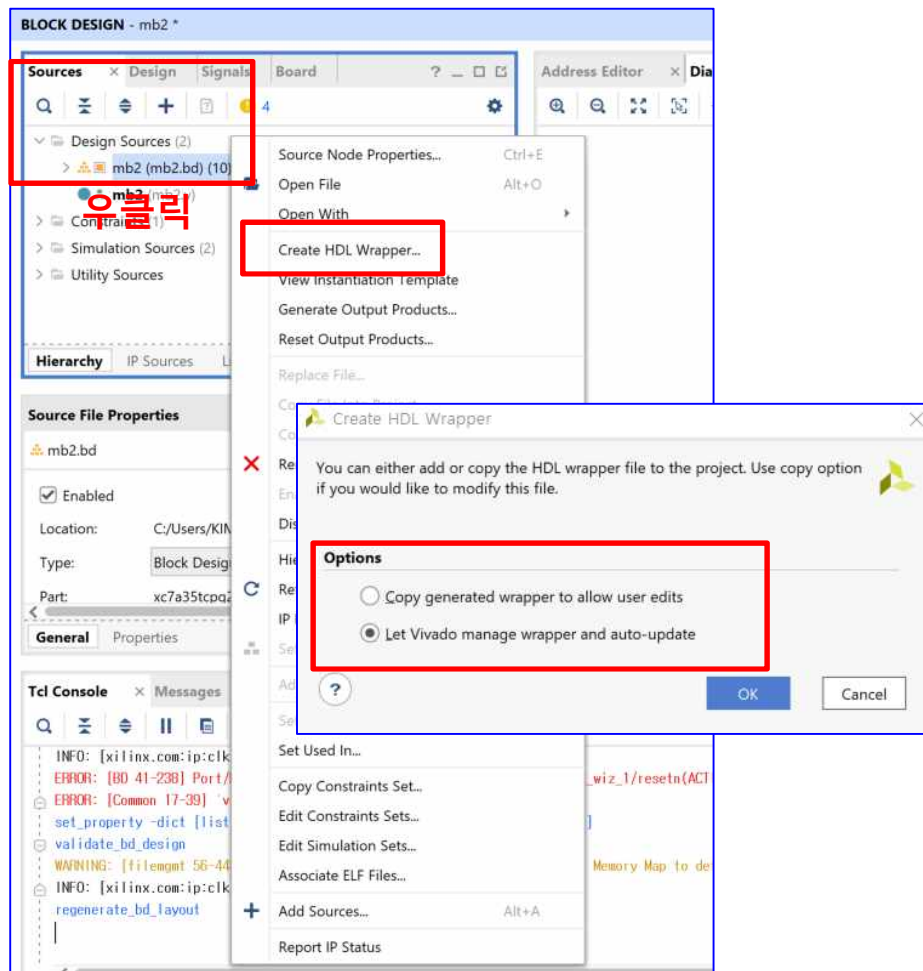


Hello World Implementation

Microblaze

➤ Hello World Implementation

- IP Block ➔ 코드 변환
- [Sources] ➔ [Design sources] ➔ [microblaze] 선택 ➔ 우클릭 ➔ [Create HDL Wrapper...] 클릭 ➔ OK



Hello World Implementation

Microblaze

➤ Hello World Implementation

- IP Block ➔ 코드 변환
- [Sources] ➔ [Design sources] ➔ [microblaze] 선택 ➔ 우클릭 ➔ [Create HDL Wrapper...] 클릭 ➔ OK

```
Address Editor  x Diagram  x mb2.v  x mb2_wrapper.v  x
c:/Users/KIM/2023_1/mb2/mb2.gen/sources_1/bd/mb2/hdl/mb2_wrapper.v

6 //Host      : DESKTOP-QVKKRPG running 64-bit major release (build 9200
7 //Command   : generate_target mb2_wrapper.bd
8 //Design    : mb2_wrapper
9 //Purpose   : IP block netlist
10 //-----
11 `timescale 1 ps / 1 ps
12
13 module mb2_wrapper
14     (resetsn,
15      sys_clock,
16      usb_uart_rxd,
17      usb_uart_txd);
18     input resetsn;
19     input sys_clock;
20     input usb_uart_rxd;
21     output usb_uart_txd;
22
23     wire resetsn;
24     wire sys_clock;
25     wire usb_uart_rxd;
26     wire usb_uart_txd;
27
28     mb2 mb2_i
29     (
30         .resetsn(resetsn),
31         .sys_clock(sys_clock),
32         .usb_uart_rxd(usb_uart_rxd),
33         .usb_uart_txd(usb_uart_txd));
34 endmodule
```

```
`timescale 1 ps / 1 ps
module mb2_wrapper
(resetsn,
sys_clock,
usb_UART_rxd,
usb_UART_txd);
input resetsn;
input sys_clock;
input usb_UART_rxd;
output usb_UART_txd;
```

```
wire resetsn;
wire sys_clock;
wire usb_UART_rxd;
wire usb_UART_txd;
```

```
mb2 mb2_i
(.resetsn(resetsn),
.sys_clock(sys_clock),
.usb_UART_rxd(usb_UART_rxd),
.usb_UART_txd(usb_UART_txd));
endmodule
```

Hello World Implementation

➤ *Hello World Implementation*

- *Create Constraint File (*.XDC)*

➔ 실습

Hello World Implementation

Microblaze

➤ Hello World Implementation

- IP → Bitstream 생성
- **PROGRAM AND DEBUG → Generate Bitstream** [→ 각 IP 코드 생성, Synthesis 및 Implementation 진행됨]

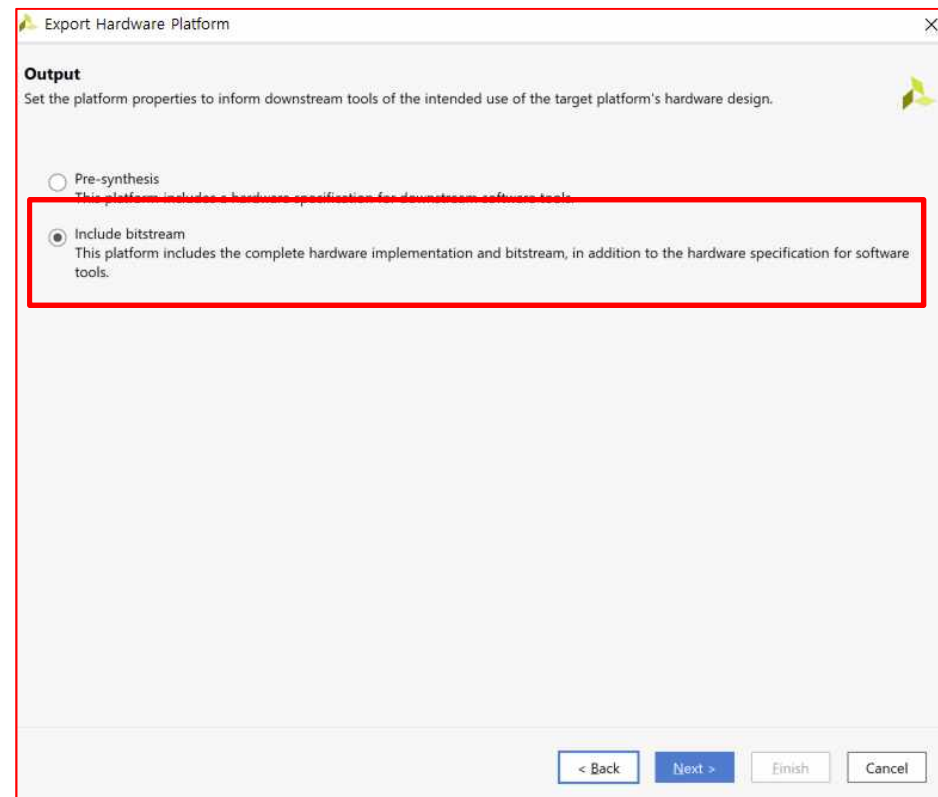
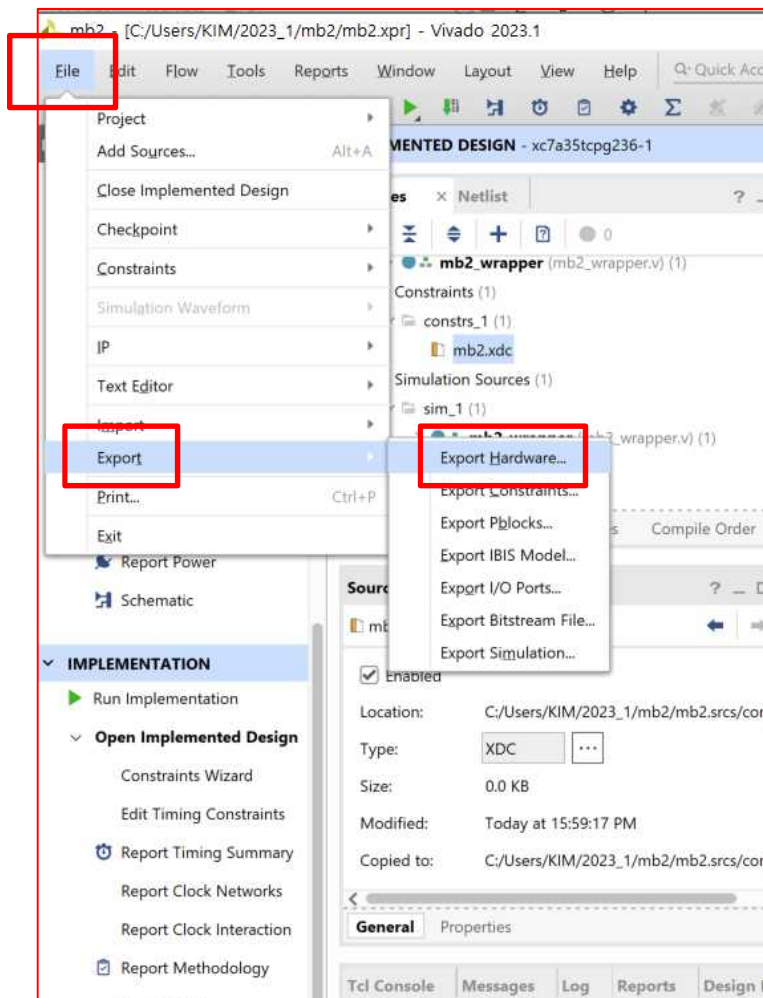
The screenshot displays the Vivado 2023.1 interface during the implementation of a Hello World project. The top-left pane shows the 'IMPLEMENTATION' tab, where the 'Generate Bitstream' option under 'PROGRAM AND DEBUG' is highlighted. A dialog box titled 'No Implementation Results Available' is open, indicating that synthesis and implementation have not yet been completed. The top-right pane shows the 'BLOCK DESIGN' tab, displaying the 'mb2_wrapper.v' component. The bottom-left pane shows the 'Schematic' tab, which contains a circuit diagram of the 'mb2_wrapper.v' component. The diagram illustrates the internal structure of the wrapper, including input buffers (IBUF) for 'resetn', 'sys_clock', and 'usb_uart_rxd', and an output buffer (OBUF) for 'usb_uart_txd'. The bottom-right pane shows the 'Source File Properties' for 'mb2_wrapper.v', indicating it is a Verilog file of size 1.0 KB. A 'Bitstream Generation Completed' dialog box is also visible, confirming the successful completion of the bitstream generation process.

Hello World Implementation

Microblaze

➤ Hello World Implementation

- IP Block ➔ 코드 변환
- [File] ➔ [Export] ➔ [Export Hardware...] 선택 ➔ **Include bitstream** ➔ Next

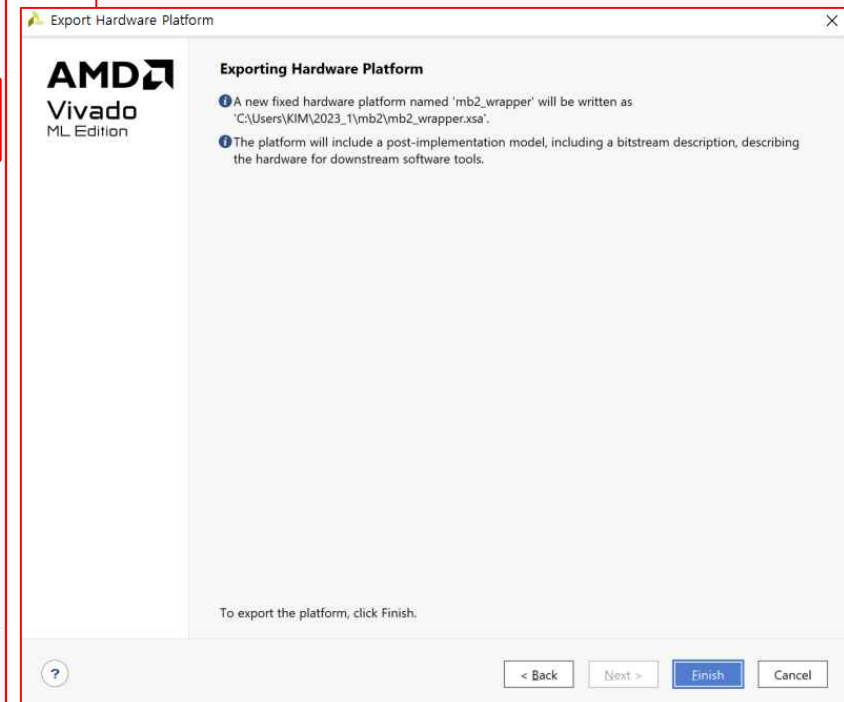
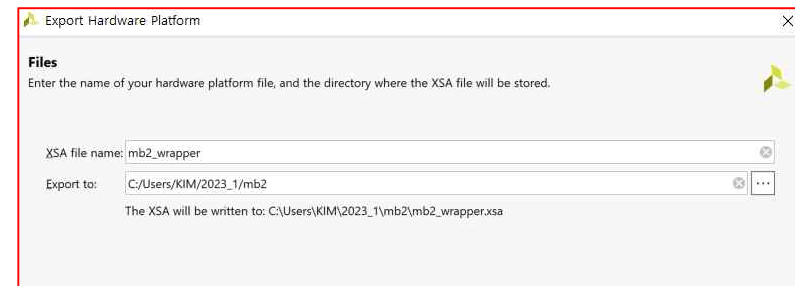
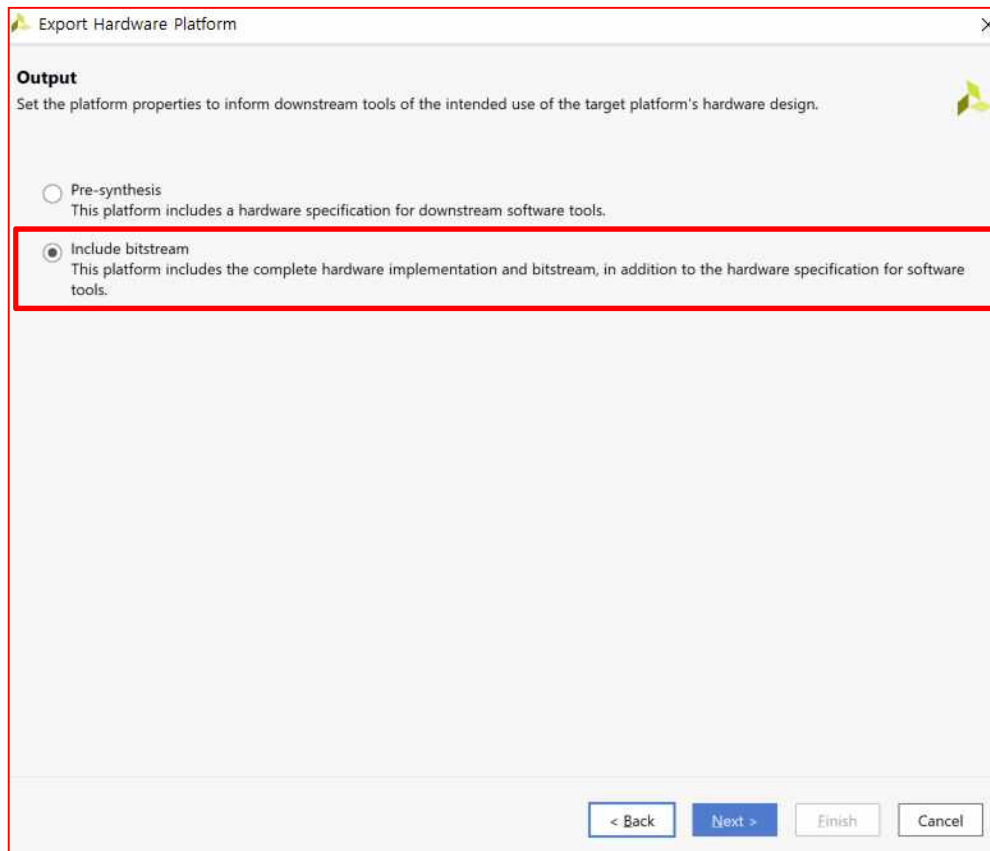


Hello World Implementation

Microblaze

➤ Hello World Implementation

- Vitis 에서 사용하기 위하여 **File** ➔ **Export - Export Hardware** 실행 ➔ “**Include bitstream**” 선택 ➔ bitstream을 포함해서 **Export**
- Bitstream을 포함해서 **.xsa** 파일 생성
- 소프트웨어 개발을 위한 **XSA** 파일 생성이 완료

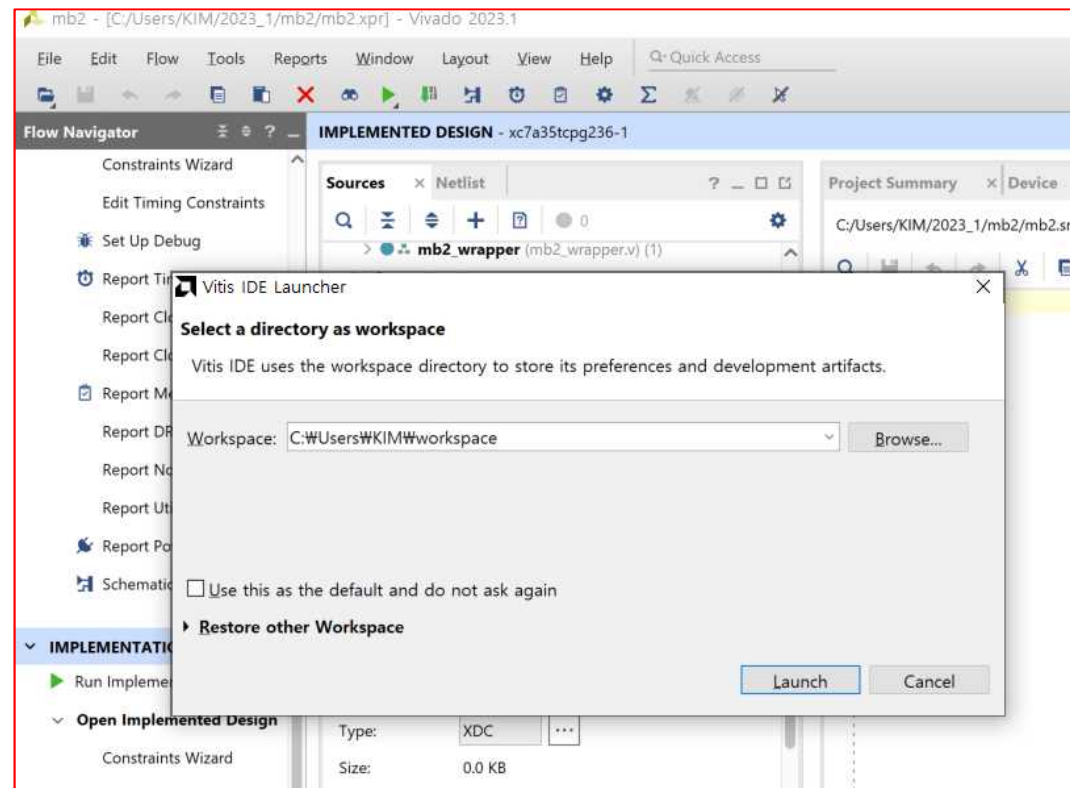
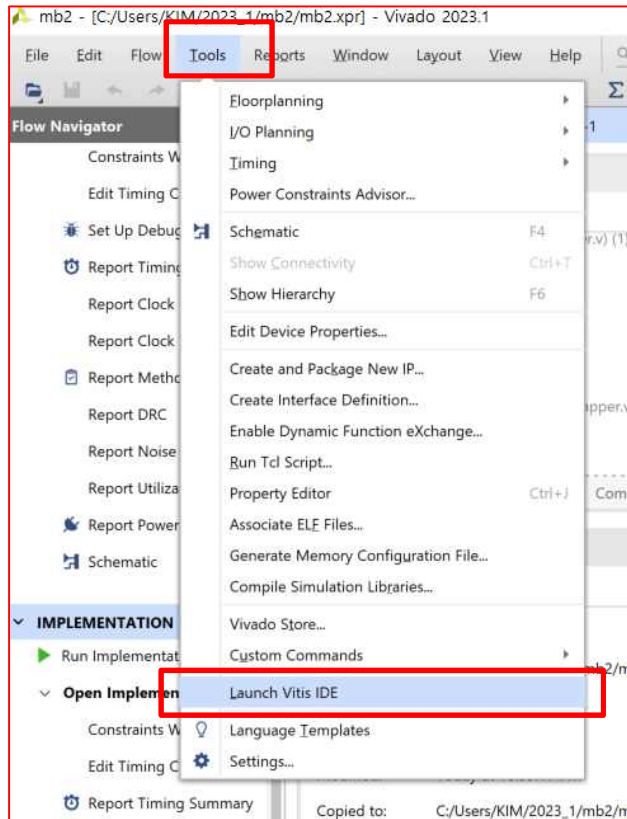


Hello World Implementation

Microblaze

➤ Hello World Implementation → Vitis Application Implementation

- Application 구현 위한 MicroBlaze 생성이 완료
- Application SW를 구현하기 위하여 Vitis를 실행 (방법1): “Vitis 2022.1(or Vitis 2023.1)” 을 바로 실행
- Application SW를 구현하기 위하여 Vitis를 실행 (방법2): Vivado 에서 Tools → Launch Vitis IDE 를 클릭
- Vitis 실행 → Workspace 폴더 지정 → Vivado에서 실행했던 폴더를 지정, HW와 SW를 같은 폴더에서 관리가 용이 → Launch 버튼을 클릭해서 Vitis를 시작



VIVADO(HW)

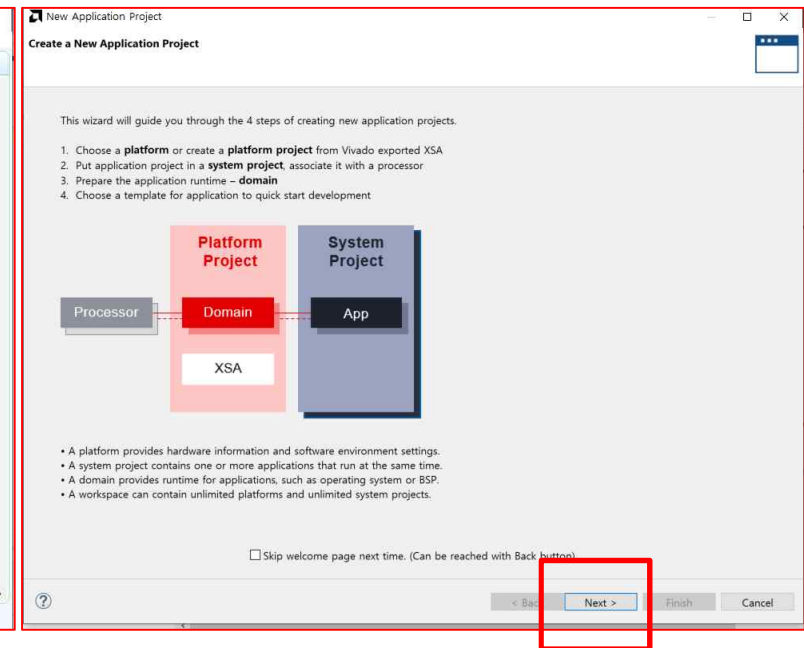
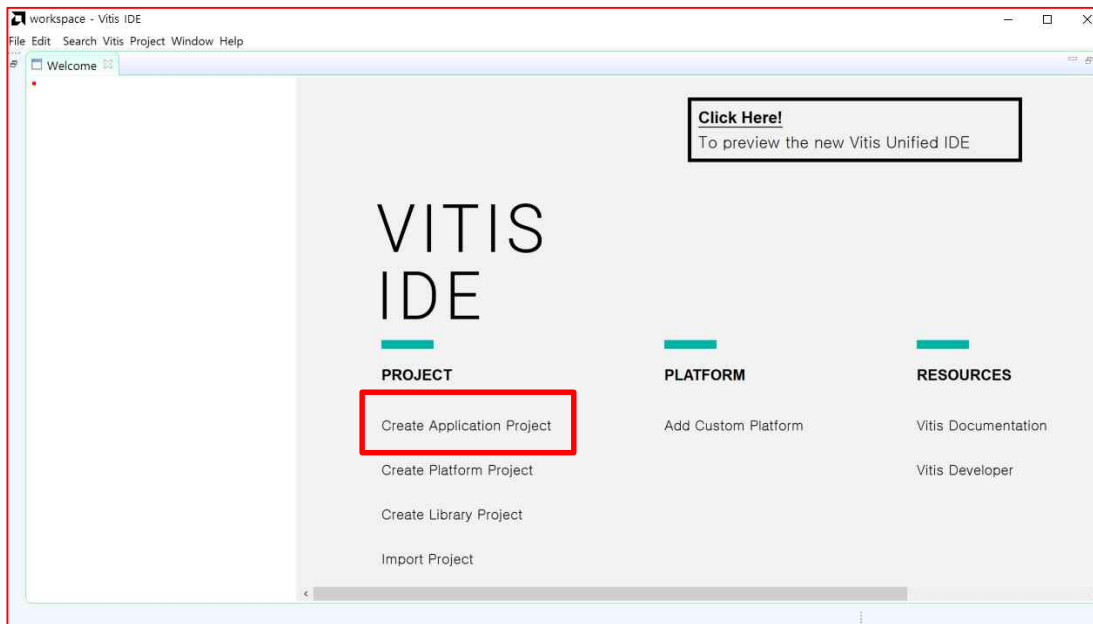
➔ *VITIS IDE(AS, Application Software)*

Hello World Implementation

Microblaze

➤ Hello World Implementation

- **Vitis** 에서 2개의 프로젝트를 생성
- **Vivado** 에서 구현한 **HW(xsa 파일)**를 설정하는 **Platform Project**
- 두번째는 **Application SW**를 구현하는 **Application Project**
- **Vitis**가 **2022.1** 버전으로 업데이트 되면서 **Application Project** 내에서 **Platform Project**를 생성가능
- “**Create Application Project**”를 클릭



Hello World Implementation

Microblaze

➤ Hello World Implementation

- New Application Project ➔ “Create a new platform from hardware (XSA)” ➔ Platform Project 생성
- *_wrapper.xsa 파일 열기

The image displays three screenshots from the Xilinx IDE illustrating the 'Hello World' implementation steps:

- Left Screenshot:** The 'New Application Project' dialog box. The 'Platform' section shows 'Create a new platform from hardware (XSA)' selected. Below, a table lists available platforms. A red box highlights the 'Create a new platform from hardware (XSA)' button.
- Middle Screenshot:** The 'Create Platform from XSA' dialog box. The 'XSA File' section shows a list of XSA files. A red box highlights the 'microblaze_0' folder. The 'Platform name' field is empty.
- Right Screenshot:** The 'Create Platform from XSA' dialog box. The 'XSA File' section shows a list of XSA files. A red box highlights the 'microblaze_0_wrapper.xsa' file. The 'Platform name' field is empty.

Additional annotations include:

- A red box around the 'New Application Project' menu item in the IDE.
- A red box around the 'microblaze_0_wrapper.xsa' file in the IDE's project browser.
- Text: "New Application Project 열리지 않을 경우 경로" (If the New Application Project does not open, use the path).

Hello World Implementation

Microblaze

➤ Hello World Implementation

- New Application Project ➔ “Create a new platform from hardware (XSA) ➔ Platform Project 생성
- [*_wrapper.xsa] 파일 열기 ➔ Next
- Application project Name 설정 : (ex) mymb0App [System project name : mymb0App_system] ➔ Next

The image displays two screenshots of the Xilinx IDE's 'New Application Project' wizard, illustrating the steps for creating a new platform from hardware (XSA).

Left Screenshot: Platform Selection

- Platform:** A note states, "A platform project will be generated automatically in workspace for the selected XSA. It can be customized later."
- Select a platform from repository:** The 'Create a new platform from hardware (XSA)' button is highlighted with a red box.
- Hardware Specification:** A list of XSA files is shown, including 'C:\Users\WKIM\2023_1\microblaze_0\microblaze_0_wrapper.xsa'.
- XSA File:** The selected file is 'C:\Users\WKIM\2023_1\microblaze_0\microblaze_0_wrapper.xsa'.
- Platform name:** The name 'microblaze_0_wrapper_1' is entered.

Right Screenshot: Application Project Details

- Application Project Details:** The title bar indicates 'Specify the application project name and its system project properties'.
- Application project name:** The name 'mymb0App' is entered and highlighted with a red box.
- System Project:** The instruction is 'Create a new system project for the application or select an existing one from the workspace'.
- System project details:** The 'System project name' is 'mymb0App_system'.
- Target processor:** The instruction is 'Select target processor for the Application project.' A table lists the processor and associated applications:

Processor	Associated applications
microblaze_0	mymb0App

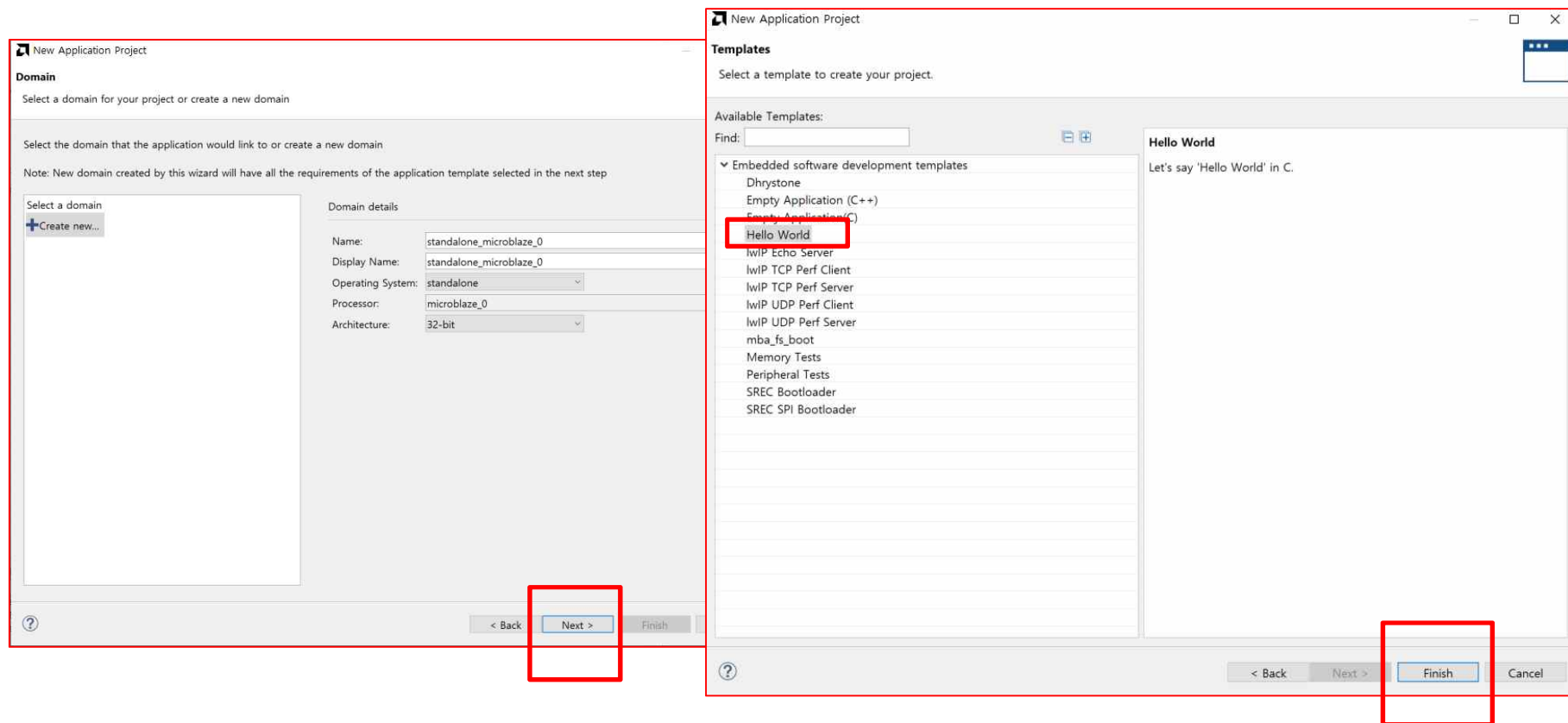
The 'Show all processors in the hardware specification' checkbox is checked.

Hello World Implementation

Microblaze

➤ Hello World Implementation

- New Application Project ➔ “Create a new platform from hardware (XSA) ➔ Platform Project 생성
- [*_wrapper.xsa] 파일 열기 ➔ Next
- Application project Name 설정 ➔ Next
- **Template ➔ Available Templates ➔ “Hello World” ➔ Finish**



Hello World Implementation

Microblaze
mb2.xpr//microblaze_0.xpr

➤ Hello World Implementation

- **helloworld.c** ➔ 클릭

The screenshot shows the Vitis IDE interface for a Microblaze project. The Explorer view on the left displays the project structure, with 'helloworld.c' highlighted under the 'src' directory. The Assistant view shows the project hierarchy, including 'microblaze_0_wrapper', 'microblaze_0_wrapper_1', 'microblazeapp_system', and 'microblazeapp'. The main editor displays the 'helloworld.c' source code, which includes headers for 'stdio.h', 'platform.h', and 'xil_printf.h', and a 'main' function that prints 'Hello World' and 'Successfully ran Hello World application'. A 'Vitis IDE Launcher' dialog box is open in the top right corner, showing the workspace directory 'C:\Users\WKIM\2023_1\microblaze_0' and buttons for 'Launch' and 'Cancel'.

```
30 *
31 *
32 *
33 /*
34  * helloworld.c: simple test application
35  *
36  * This application configures UART 16550 to baud rate 9600.
37  * PS7 UART (Zynq) is not initialized by this application, since
38  * bootrom/bsp configures it to baud rate 115200
39  *
40  * -----
41  * | UART TYPE   BAUD RATE |
42  * -----
43  * uarts550     9600
44  * uartlite     Configurable only in HW design
45  * ps7_uart     115200 (configured by bootrom/bsp)
46  */
47
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51
52
53 int main()
54 {
55     init_platform();
56
57     print("Hello World\n\r");
58     print("Successfully ran Hello World application");
59     cleanup_platform();
60     return 0;
61 }
62
```

Hello World Implementation

Microblaze
mb2.xpr/microblaze_0.xpr

➤ Hello World Implementation

- *helloworld.c* ➔ 클릭
- *UART TYPE : UARTns550 ➔ BAUD RATE : 9,600*

The screenshot displays the Microblaze IDE interface. The Explorer pane on the left shows the project structure, with the `helloworld.c` file highlighted under the `microblazeapp_system` project. The main editor window shows the source code of `helloworld.c`, which includes comments about the UART configuration and the implementation of the `main` function. The `main` function calls `init_platform()`, prints "Hello World\n\r", and prints "Successfully ran Hello World application". The ComPortMaster utility is open in the bottom right, showing the port configuration for COM6, baud rate 9600, and data bits 8. The utility is used to send data to the device.

```
30 *
31 *
32 *
33 /*
34  * helloworld.c: simple test application
35  *
36  * This application configures UART 16550 to baud rate 9600.
37  * PS7 UART (Zynq) is not initialized by this application, since
38  * bootrom/bsp configures it to baud rate 115200
39  */
40
41 * | UART TYPE  BAUD RATE |
42 *
43 * uartns550   9600
44 * uartrlite   Configurable only in HW design
45 * ps7_uart    115200 (configured by bootrom/bsp)
46 */
47
48 #include <stdio.h>
49 #include "platform.h"
50 #include "xil_printf.h"
51
52
53 int main()
54 {
55     init_platform();
56
57     print("Hello World\n\r");
58     print("Successfully ran Hello World application");
59     cleanup_platform();
60     return 0;
61 }
62
```

ComPortMaster - Withrobot Co.

Port Config

Device: COM6

Baudrate: 9600

Data bits: 8

Stop bits: 1

Parity: None

Quick Send

Send Multiple: 0 / 1

Interval: 100 ms

Send

Recv

Decode SLIP

Auto CR/LF

Handle CR/LF

Start Capture

Clear

ComPortMaster

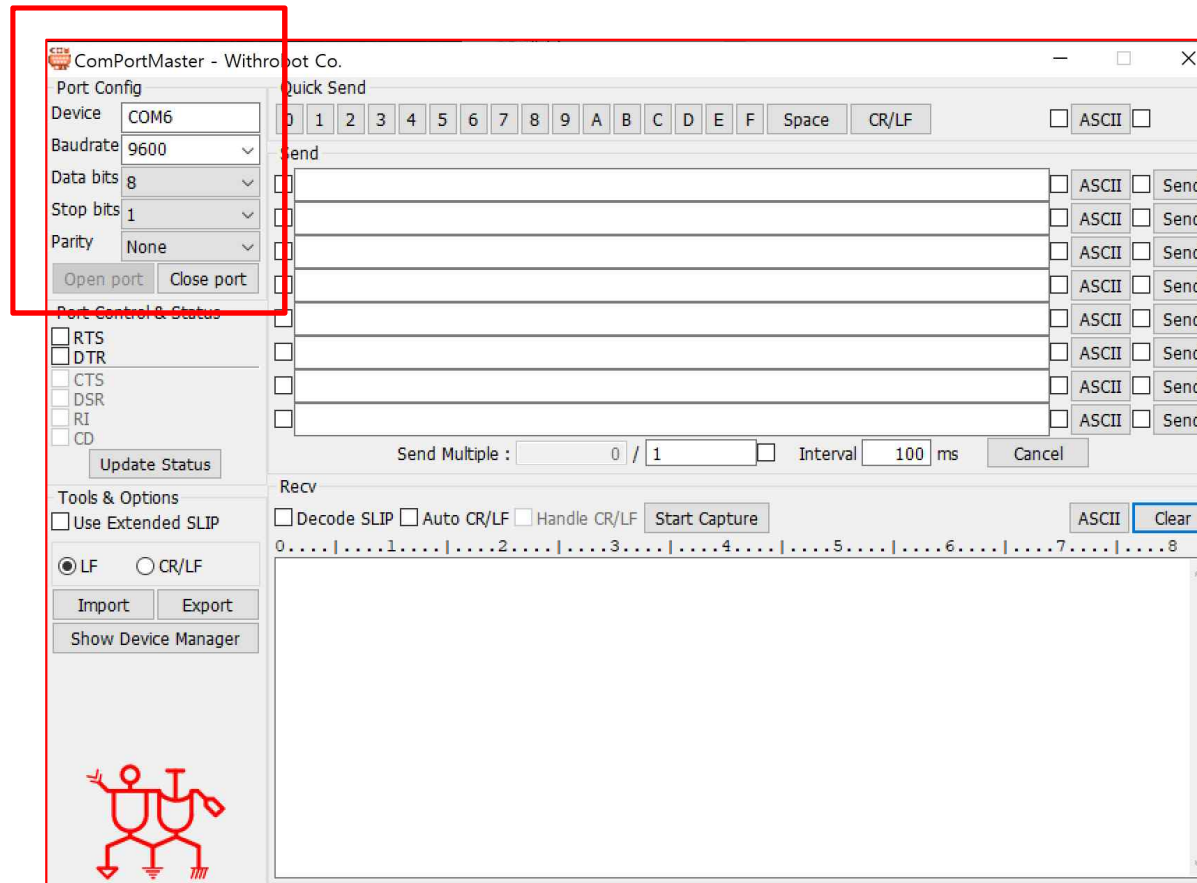
<http://withrobot.com/data/?mod=document&uid=12>

Hello World Implementation

Microblaze
mb2.xpr//microblaze_0.xpr

➤ Hello World Implementation

- ComPortMaster : 직렬(시리얼)통신 터미널 프로그램 [<http://withrobot.com/data/?mod=document&uid=12>]



Hello World Implementation

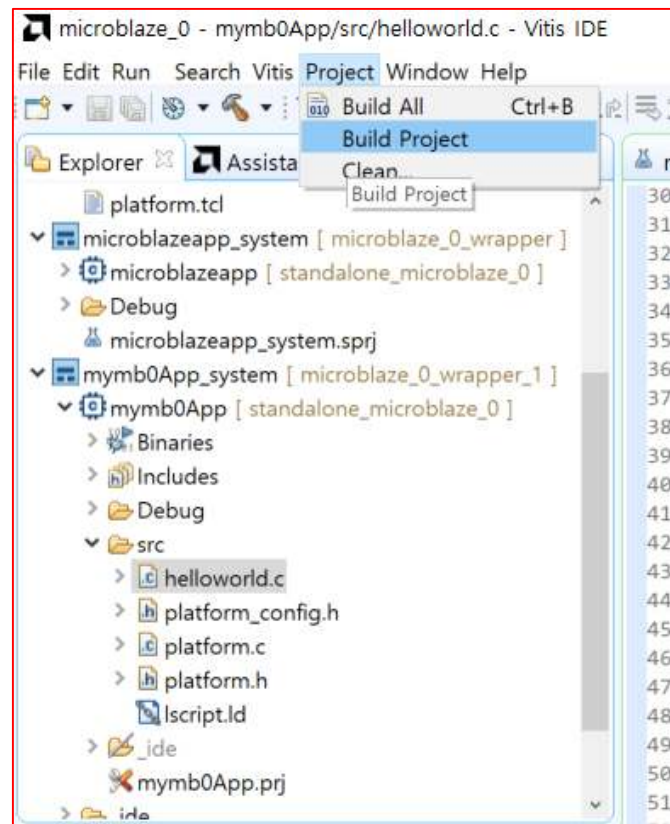
Microblaze
mb2.xpr/microblaze_0.xpr

➤ Hello World Implementation

- **Project → Build Project → Program Device**
- **Vitis → Program Device → Browse →**

C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf → 선택 , 열기 → Program

Project → Build Project



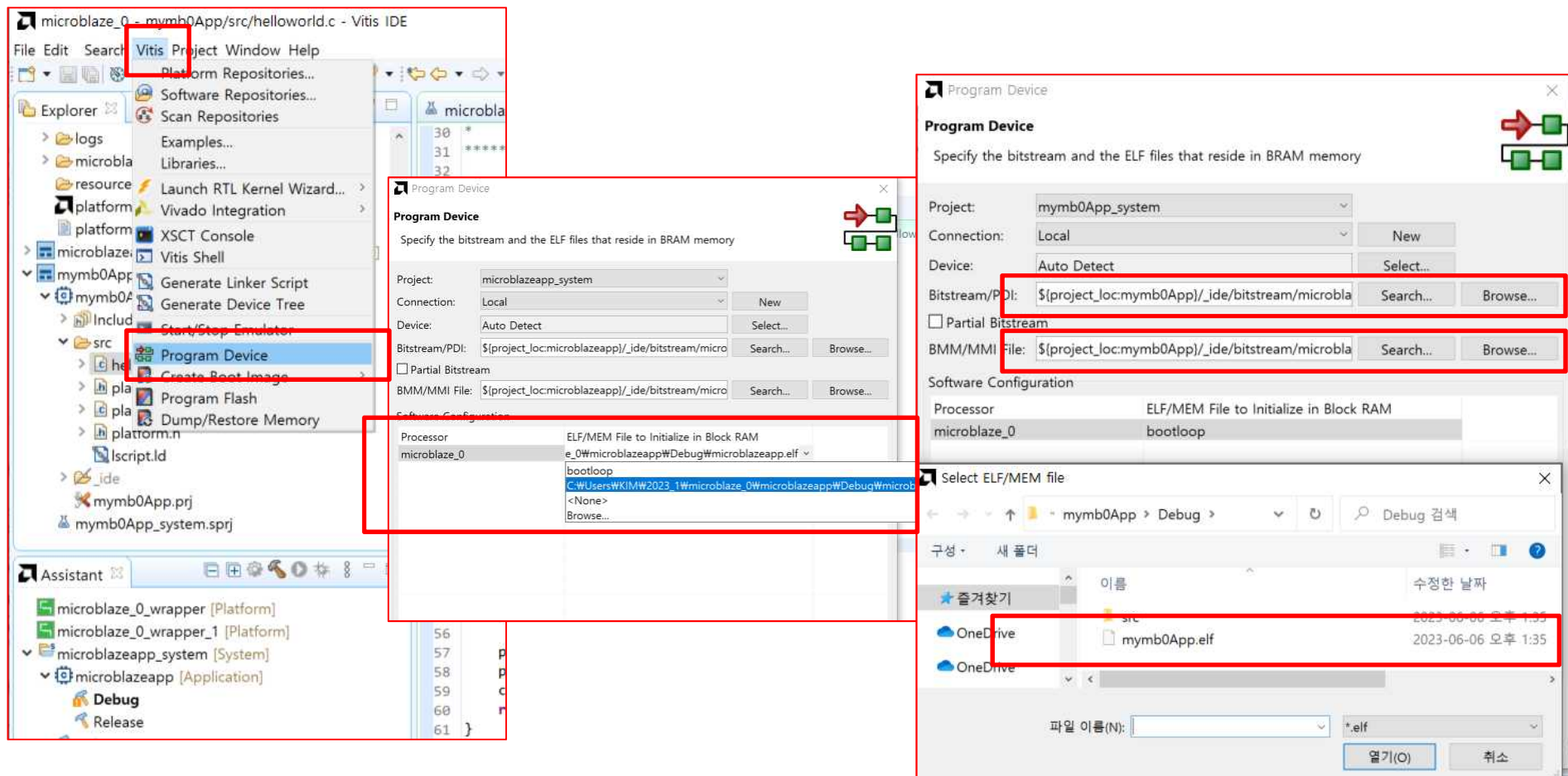
Hello World Implementation

Microblaze

➤ Hello World Implementation

- Vitis ➔ Program Device ➔ Browse ➔

C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf ➔ 선택, 열기 ➔ Program



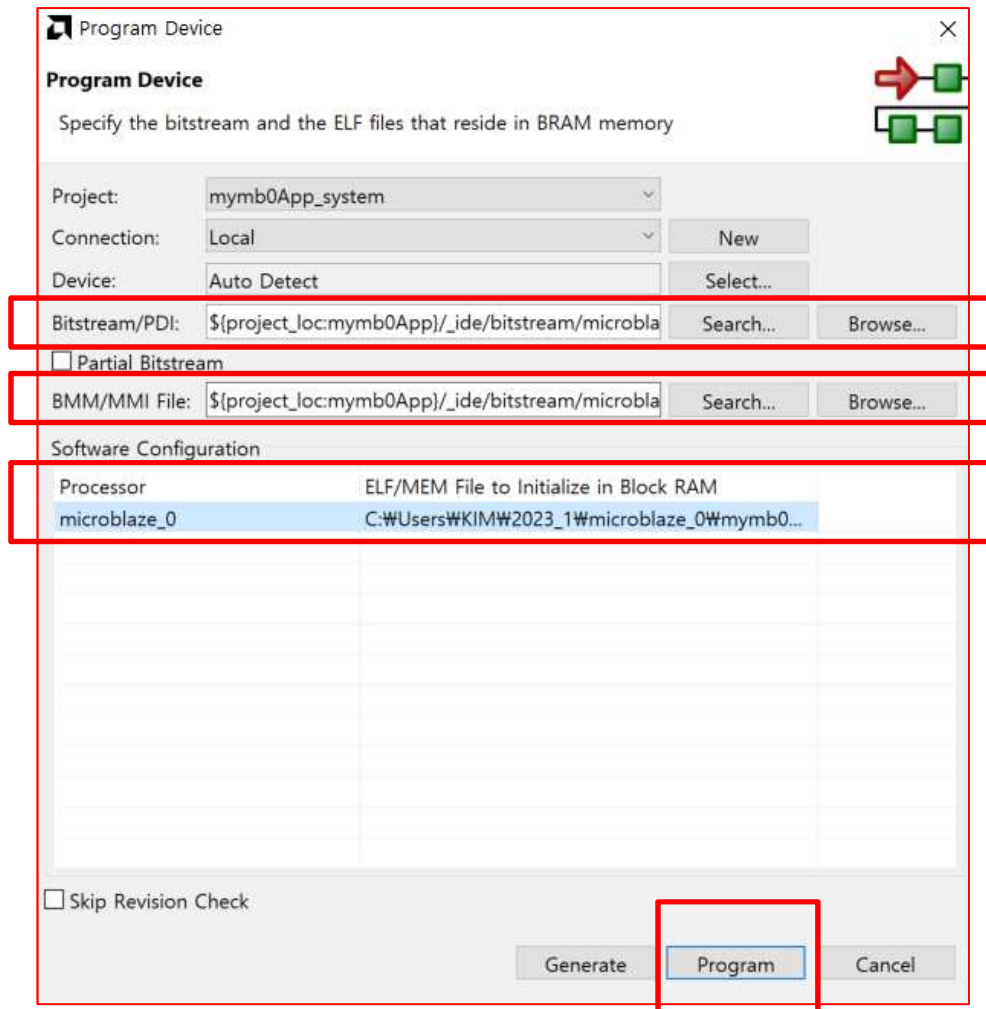
Hello World Implementation

Microblaze

➤ Hello World Implementation

- Vitis ➔ Program Device ➔ Browse ➔

C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf ➔ 선택, 열기 ➔ **Program**



Hello World Implementation

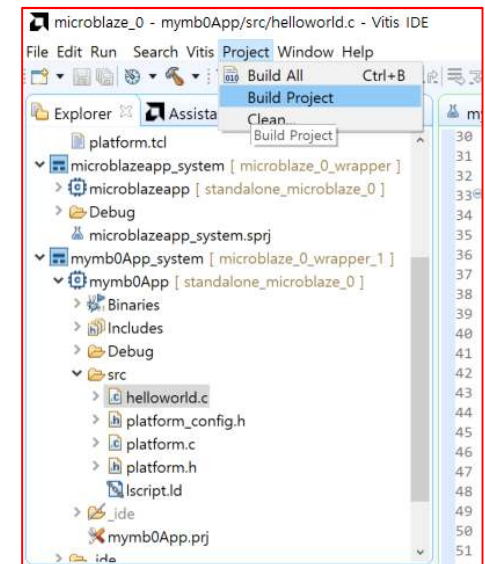
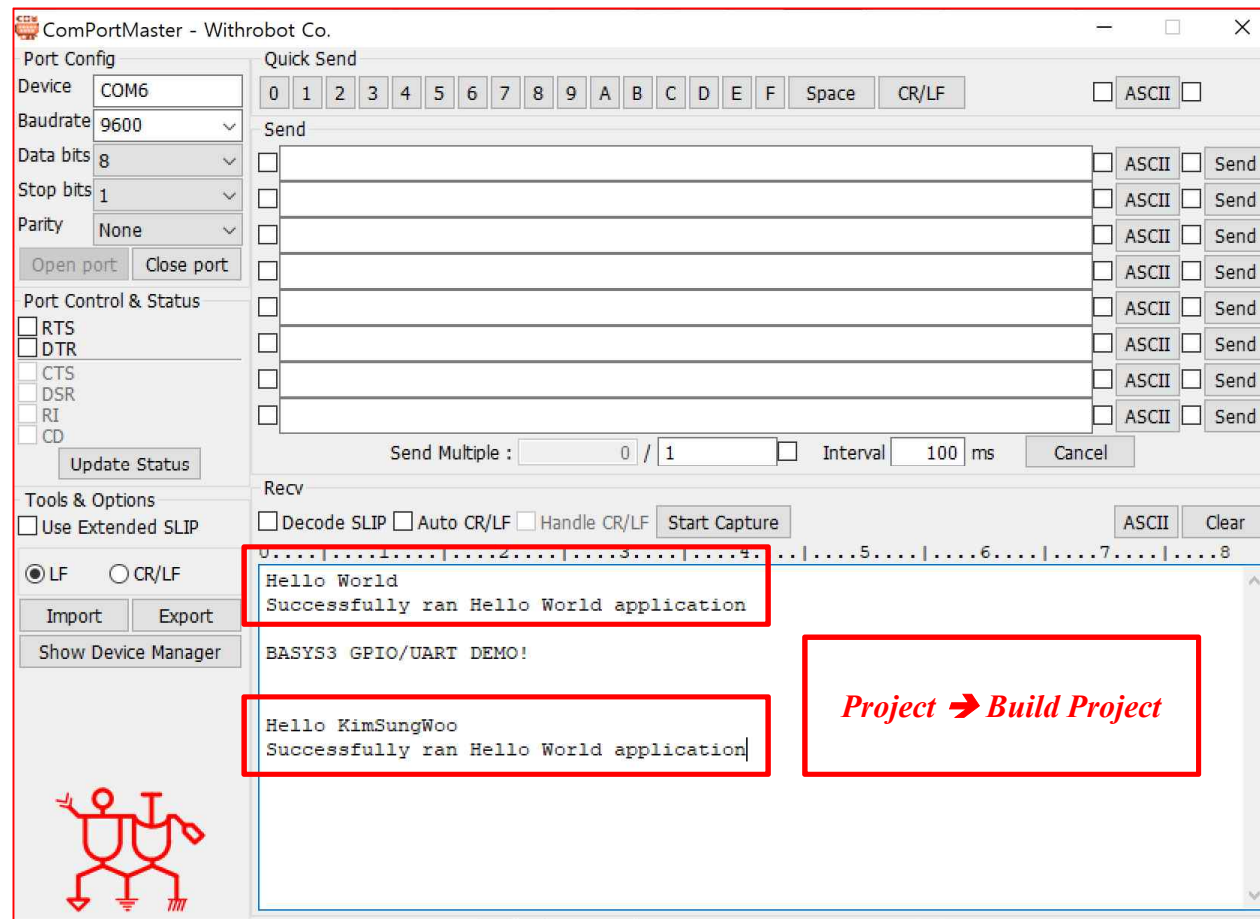
Microblaze
mb2.xpr/microblaze_0.xpr

➤ Hello World Implementation

- Vitis ➔ Program Device ➔ Browse ➔

C:\Users\KIM\2023_1\microblaze_0\mymb0App\Debug\mymb0App.elf ➔ 선택 , 열기 ➔ Program

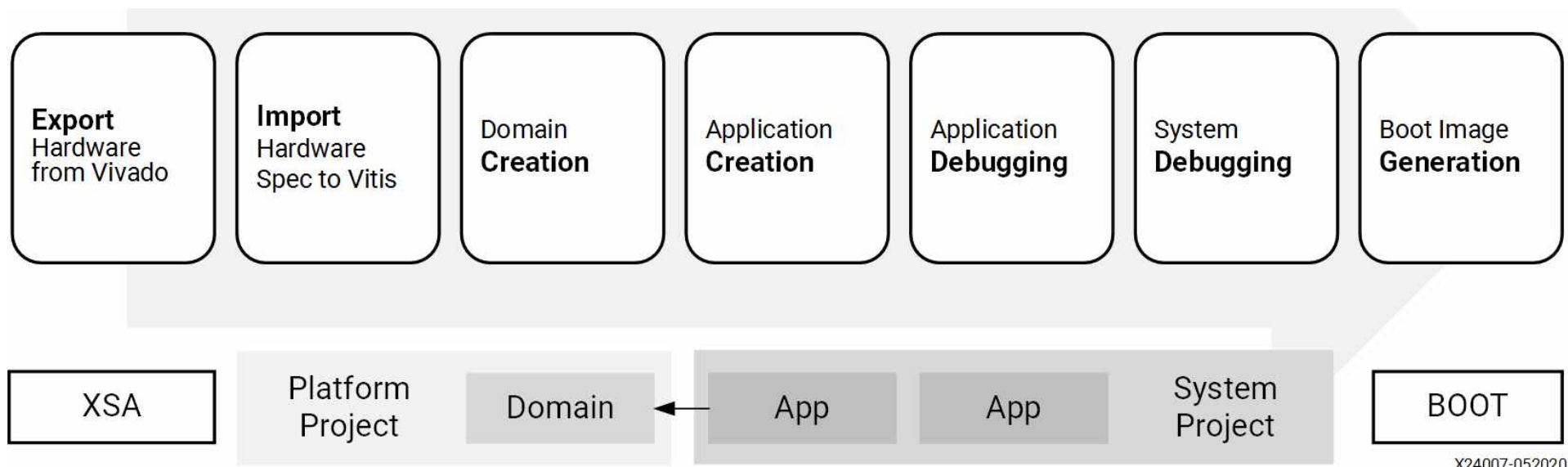
- Project ➔ Build Project ➔ Program Device ➔ **Hello World Implementation Result Check using UART**



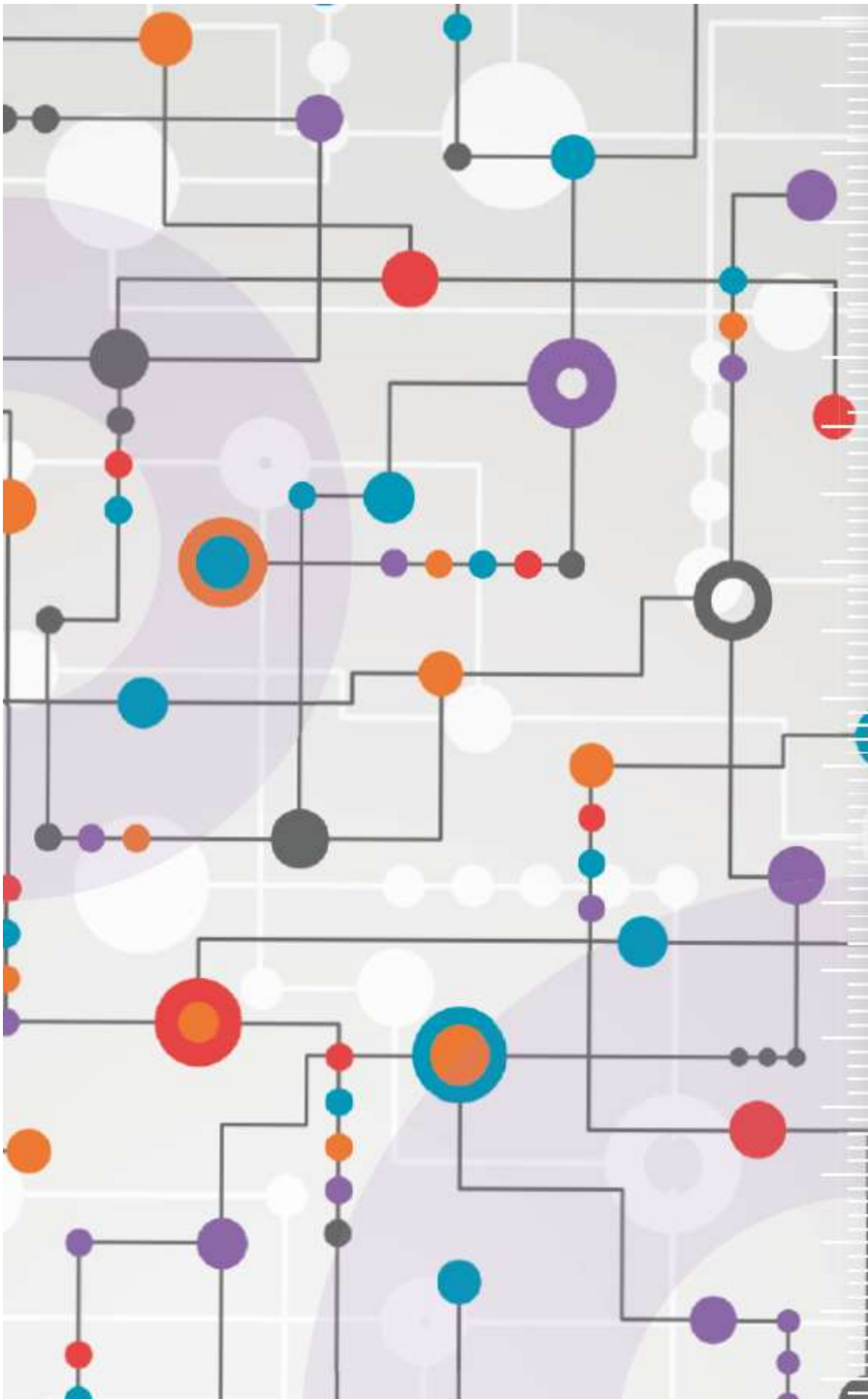
Hello World Implementation

Microblaze

- **VITIS** 소프트웨어 구현 [<https://docs.xilinx.com/r/en-US/ug1400-vitis-embedded/Getting-Started-with-Vitis>]
 - **VIVADO**를 이용해서 **xsa** 확장자인 하드웨어(HW)를 만들고 이 파일을 바탕으로 **VITIS**에서 사용 할 수 있는 도메인(**Domain**)을 생성 →도메인(**Domain**)을 이용해서 소프트웨어(**SW**)를 생성하고 디버깅 (**Dubugging**)을 진행
 - **Figure: Embedded Software Application Development Workflow**



- **VITIS**를 이용한 전체 소프트웨어 구현 프로세스



수고하셨습니다.